

Machine Learning - Python Variables, Transformaciones, Segmentacion, Descriptores

July 20, 2026



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub * URL: <https://github.com/Exosphere-Idyllic/Machine-Learning>

1 3_1_3_var_cualitativas

1.1 01PAO26-26 - Variables cualitativas

```
[1]: # Creación aleatoria de datos
import numpy as np
import random
random.seed(42)
```

```
[2]: nivel_estudios = []
nivel_estudios.extend(['Secundaria']*110)
nivel_estudios.extend(['Bachillerato']*172)
nivel_estudios.extend(['Grado']*145)
nivel_estudios.extend(['Master']*52)
nivel_estudios.extend(['Doctorado']*21)

nivel_estudios = np.random.permutation(nivel_estudios)
```

```
[3]: print(nivel_estudios)
```

```
['Bachillerato' 'Grado' 'Secundaria' 'Grado' 'Secundaria' 'Master' 'Grado'
 'Grado' 'Secundaria' 'Grado' 'Secundaria' 'Bachillerato' 'Grado' 'Grado'
```

'Bachillerato' 'Grado' 'Grado' 'Grado' 'Grado' 'Bachillerato' 'Grado'
 'Bachillerato' 'Grado' 'Grado' 'Secundaria' 'Secundaria' 'Bachillerato'
 'Grado' 'Grado' 'Master' 'Master' 'Bachillerato' 'Grado' 'Secundaria'
 'Master' 'Grado' 'Bachillerato' 'Bachillerato' 'Grado' 'Master'
 'Bachillerato' 'Secundaria' 'Bachillerato' 'Grado' 'Bachillerato'
 'Bachillerato' 'Bachillerato' 'Grado' 'Grado' 'Bachillerato' 'Secundaria'
 'Grado' 'Bachillerato' 'Grado' 'Grado' 'Secundaria' 'Grado' 'Grado'
 'Grado' 'Bachillerato' 'Master' 'Bachillerato' 'Doctorado' 'Grado'
 'Bachillerato' 'Bachillerato' 'Master' 'Bachillerato' 'Grado' 'Grado'
 'Secundaria' 'Bachillerato' 'Bachillerato' 'Grado' 'Secundaria'
 'Secundaria' 'Secundaria' 'Master' 'Doctorado' 'Bachillerato' 'Grado'
 'Grado' 'Grado' 'Grado' 'Grado' 'Secundaria' 'Secundaria' 'Doctorado'
 'Secundaria' 'Grado' 'Master' 'Bachillerato' 'Bachillerato' 'Grado'
 'Bachillerato' 'Secundaria' 'Secundaria' 'Bachillerato' 'Bachillerato'
 'Doctorado' 'Grado' 'Bachillerato' 'Secundaria' 'Grado' 'Secundaria'
 'Secundaria' 'Master' 'Bachillerato' 'Bachillerato' 'Grado'
 'Bachillerato' 'Secundaria' 'Bachillerato' 'Master' 'Bachillerato'
 'Secundaria' 'Secundaria' 'Grado' 'Grado' 'Grado' 'Secundaria' 'Master'
 'Master' 'Grado' 'Bachillerato' 'Master' 'Secundaria' 'Secundaria'
 'Master' 'Grado' 'Grado' 'Secundaria' 'Bachillerato' 'Grado' 'Secundaria'
 'Secundaria' 'Grado' 'Grado' 'Grado' 'Master' 'Secundaria' 'Master'
 'Secundaria' 'Secundaria' 'Doctorado' 'Bachillerato' 'Grado' 'Secundaria'
 'Bachillerato' 'Bachillerato' 'Secundaria' 'Bachillerato' 'Bachillerato'
 'Master' 'Doctorado' 'Secundaria' 'Grado' 'Grado' 'Bachillerato'
 'Bachillerato' 'Secundaria' 'Secundaria' 'Secundaria' 'Bachillerato'
 'Bachillerato' 'Grado' 'Doctorado' 'Master' 'Secundaria' 'Secundaria'
 'Bachillerato' 'Grado' 'Doctorado' 'Bachillerato' 'Bachillerato'
 'Secundaria' 'Grado' 'Secundaria' 'Bachillerato' 'Grado' 'Bachillerato'
 'Grado' 'Grado' 'Bachillerato' 'Bachillerato' 'Secundaria' 'Master'
 'Grado' 'Grado' 'Bachillerato' 'Secundaria' 'Secundaria' 'Secundaria'
 'Grado' 'Grado' 'Bachillerato' 'Grado' 'Secundaria' 'Secundaria'
 'Secundaria' 'Bachillerato' 'Secundaria' 'Bachillerato' 'Bachillerato'
 'Secundaria' 'Grado' 'Bachillerato' 'Bachillerato' 'Master'
 'Bachillerato' 'Secundaria' 'Secundaria' 'Grado' 'Bachillerato' 'Grado'
 'Bachillerato' 'Bachillerato' 'Secundaria' 'Master' 'Grado'
 'Bachillerato' 'Bachillerato' 'Grado' 'Grado' 'Secundaria' 'Bachillerato'
 'Secundaria' 'Grado' 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Grado'
 'Grado' 'Secundaria' 'Doctorado' 'Bachillerato' 'Bachillerato'
 'Bachillerato' 'Grado' 'Bachillerato' 'Secundaria' 'Bachillerato'
 'Secundaria' 'Bachillerato' 'Grado' 'Grado' 'Bachillerato' 'Secundaria'
 'Bachillerato' 'Bachillerato' 'Master' 'Grado' 'Grado' 'Bachillerato'
 'Secundaria' 'Bachillerato' 'Doctorado' 'Bachillerato' 'Grado'
 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Grado' 'Bachillerato'
 'Doctorado' 'Grado' 'Bachillerato' 'Grado' 'Secundaria' 'Grado'
 'Bachillerato' 'Bachillerato' 'Grado' 'Bachillerato' 'Bachillerato'
 'Secundaria' 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Secundaria'
 'Secundaria' 'Master' 'Bachillerato' 'Bachillerato' 'Grado'
 'Bachillerato' 'Grado' 'Master' 'Grado' 'Doctorado' 'Grado' 'Grado'

```

'Bachillerato' 'Secundaria' 'Bachillerato' 'Bachillerato' 'Master'
'Bachillerato' 'Bachillerato' 'Bachillerato' 'Grado' 'Master' 'Doctorado'
'Bachillerato' 'Grado' 'Bachillerato' 'Bachillerato' 'Bachillerato'
'Grado' 'Master' 'Grado' 'Secundaria' 'Bachillerato' 'Secundaria'
'Bachillerato' 'Bachillerato' 'Secundaria' 'Secundaria' 'Master'
'Bachillerato' 'Bachillerato' 'Bachillerato' 'Doctorado' 'Master' 'Grado'
'Bachillerato' 'Master' 'Grado' 'Grado' 'Bachillerato' 'Grado'
'Bachillerato' 'Bachillerato' 'Grado' 'Grado' 'Secundaria' 'Master'
'Master' 'Secundaria' 'Bachillerato' 'Grado' 'Grado' 'Bachillerato'
'Master' 'Secundaria' 'Grado' 'Master' 'Grado' 'Grado' 'Secundaria'
'Bachillerato' 'Master' 'Grado' 'Bachillerato' 'Master' 'Secundaria'
'Grado' 'Master' 'Secundaria' 'Grado' 'Bachillerato' 'Secundaria'
'Secundaria' 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Bachillerato'
'Grado' 'Bachillerato' 'Secundaria' 'Secundaria' 'Bachillerato'
'Secundaria' 'Doctorado' 'Grado' 'Grado' 'Bachillerato' 'Secundaria'
'Doctorado' 'Grado' 'Master' 'Doctorado' 'Grado' 'Master' 'Grado'
'Bachillerato' 'Bachillerato' 'Bachillerato' 'Secundaria' 'Grado' 'Grado'
'Master' 'Master' 'Grado' 'Bachillerato' 'Secundaria' 'Bachillerato'
'Bachillerato' 'Bachillerato' 'Secundaria' 'Bachillerato' 'Bachillerato'
'Grado' 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Secundaria' 'Grado'
'Bachillerato' 'Secundaria' 'Bachillerato' 'Bachillerato' 'Grado'
'Secundaria' 'Grado' 'Master' 'Secundaria' 'Bachillerato' 'Grado'
'Bachillerato' 'Bachillerato' 'Grado' 'Secundaria' 'Secundaria'
'Secundaria' 'Master' 'Secundaria' 'Grado' 'Grado' 'Master' 'Master'
'Secundaria' 'Bachillerato' 'Doctorado' 'Secundaria' 'Bachillerato'
'Bachillerato' 'Master' 'Grado' 'Bachillerato' 'Bachillerato'
'Bachillerato' 'Grado' 'Secundaria' 'Bachillerato' 'Bachillerato'
'Secundaria' 'Master' 'Grado' 'Grado' 'Bachillerato' 'Secundaria'
'Secundaria' 'Secundaria' 'Grado' 'Grado' 'Bachillerato' 'Doctorado'
'Master' 'Master' 'Grado' 'Grado' 'Bachillerato' 'Secundaria' 'Grado'
'Grado' 'Bachillerato' 'Secundaria' 'Grado' 'Bachillerato' 'Master'
'Bachillerato' 'Bachillerato' 'Secundaria' 'Secundaria' 'Bachillerato'
'Bachillerato' 'Grado' 'Bachillerato' 'Grado' 'Doctorado' 'Secundaria'
'Grado' 'Bachillerato' 'Master' 'Grado' 'Secundaria' 'Grado' 'Grado'
'Secundaria' 'Bachillerato' 'Bachillerato' 'Bachillerato' 'Doctorado'
'Bachillerato' 'Bachillerato' 'Bachillerato' 'Secundaria' 'Grado' 'Grado']

```

[4]: *# DIAGRAMA DE BARRAS*

```

import matplotlib.pyplot as plt
import numpy as np

```

```

estudios, frecuencia = np.unique(nivel_estudios, return_counts=True)
print(estudios, frecuencia)

```

```

['Bachillerato' 'Doctorado' 'Grado' 'Master' 'Secundaria'] [172  21 145  52 110]

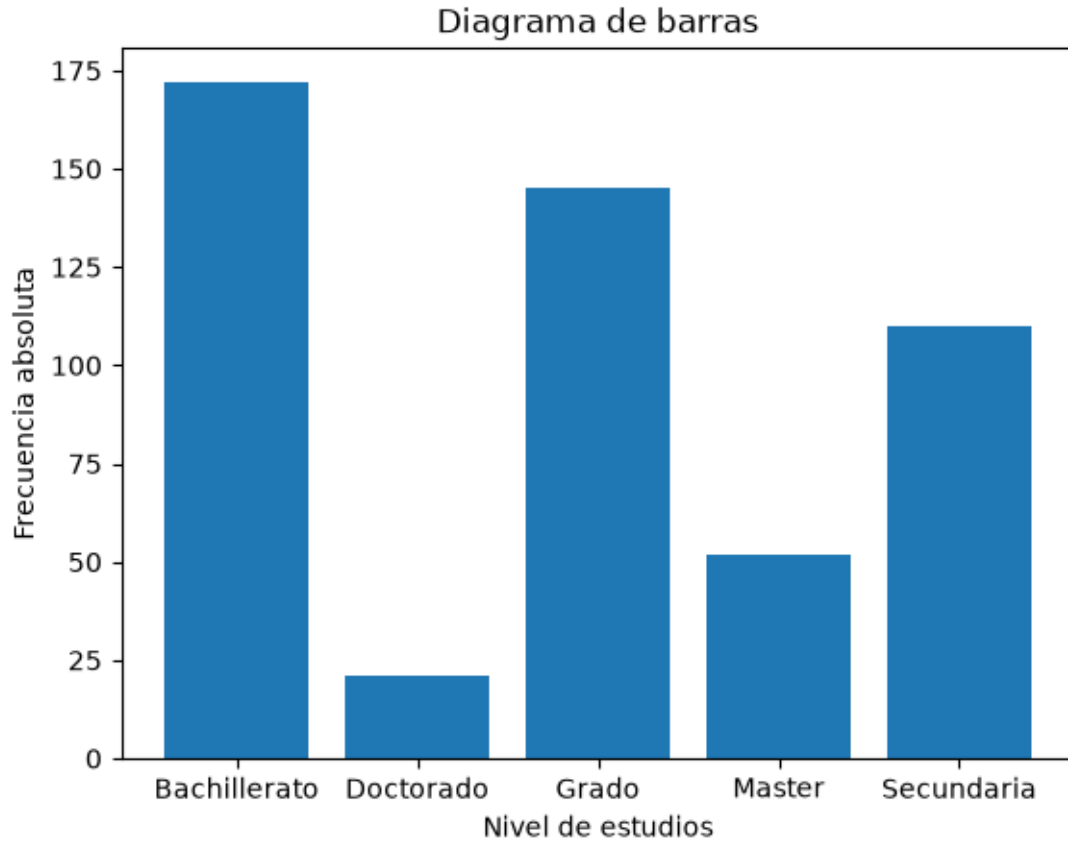
```

```

[5]: plt.ylabel('Frecuencia absoluta')
plt.xlabel('Nivel de estudios')

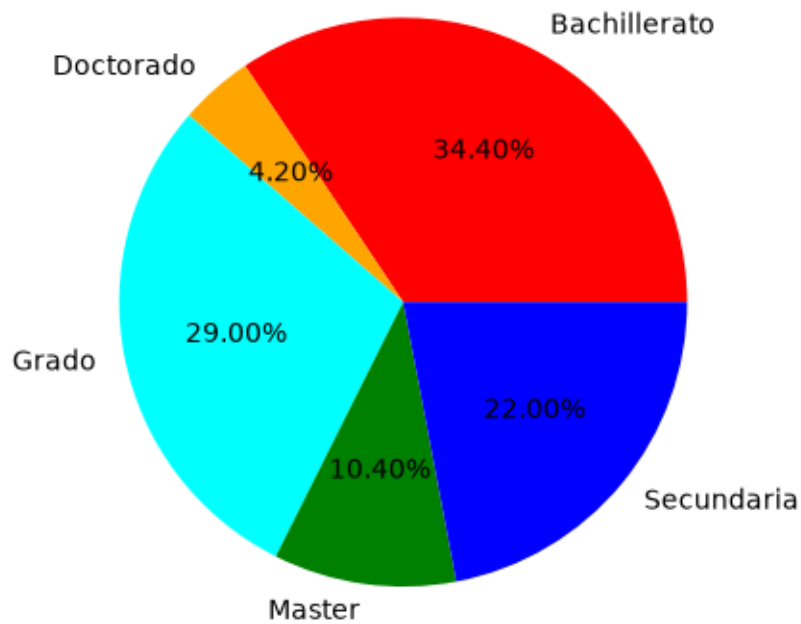
```

```
plt.title('Diagrama de barras')
plt.bar(estudios, frecuencia) # https://matplotlib.org/stable/api/\_as\_gen/matplotlib.pyplot.bar.html
plt.show()
```



```
[6]: # DIAGRAMA DE SECTORES
colores = {'red', 'blue', 'orange', 'green', 'cyan'}
plt.pie(frecuencia, labels=estudios, colors=colores, autopct='%1.2f%%')
plt.title('Diagrama de sectores')
plt.show()
```

Diagrama de sectores



```
[7]: # Tarea implementar con pandas esta información
```

2 3_1_4_var_cuantitativas

2.1 01PAO26-26 - Variables cualitativas

Elvis Pachacama

```
[8]: # Creación aleatoria de datos
import numpy as np
X = np.array([0.5, 23, 0.3, 4.5, 0.3, 0.5,
              -28, -50, 60, -100, -10, -11, 13, 19, 1, 9])
print(X)
```

```
[  0.5  23.   0.3  4.5   0.3  0.5 -28.  -50.  60. -100.
 -10.  -11.  13.  19.   1.   9. ]
```

```
[9]: # Medidas de tendencia central
from scipy import stats
media = np.mean(X)
mediana = np.median(X)
moda, _ = stats.mode(X)
print('media: ', media)
```

```
print('mediana: ', mediana)
print('moda :', moda)
```

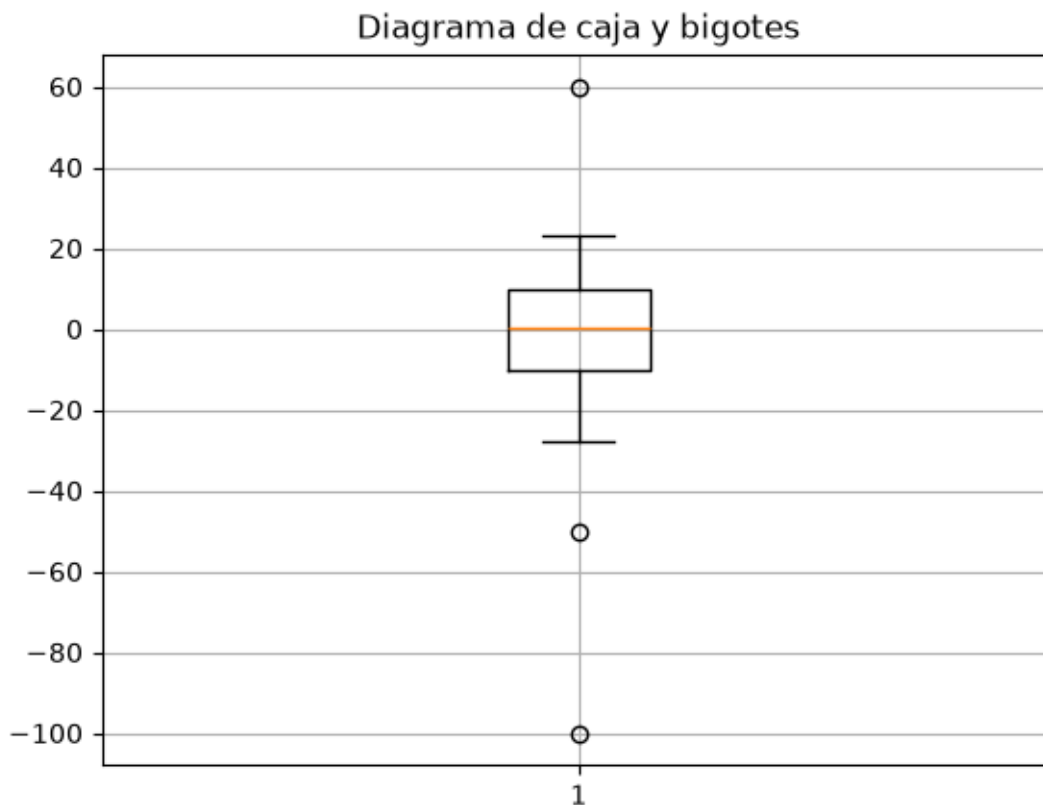
```
media:  -4.24375
mediana:  0.5
moda :  0.3
```

```
[10]: # Medidas de posición
import numpy as np
import matplotlib.pyplot as plt

Q1 = np.percentile(X, 25)
Q3 = np.percentile(X, 75)
RIC = Q3 - Q1
lim_inf = Q1 - 1.5*RIC
lim_sup = Q3 + 1.5*RIC
print('RIC :', RIC)
print('limite inferior :', lim_inf)
print('limite superior :', lim_sup)
```

```
RIC : 20.25
limite inferior : -40.625
limite superior : 40.375
```

```
[11]: # Dibuja el diagrama de box and whiskers y determina visualmente el número de
      ↪outliers
plt.boxplot(X)
plt.grid()
plt.title("Diagrama de caja y bigotes")
plt.show()
```



```
[12]: # Medidas de dispersión
desvest = np.std(X)
print('Desviación típica: ', desvest)
print('Varianza: ', desvest**2)
```

```
Desviación típica: 33.521242681880096
Varianza: 1123.6737109375
```

```
[13]: # Medidas de distribución
asimetria = stats.skew(X)
kurtosis = stats.kurtosis(X, fisher=True) # https://docs.scipy.org/doc/scipy/
                                             ↳reference/generated/scipy.stats.kurtosis.html
print('Asimetría ', np.round(asimetria, 4))
print('Curtosis ', np.round(kurtosis, 4))

if kurtosis > 0:
    print('Distribución Leptocúrtica')
elif kurtosis < 0:
    print('Distribución Platicúrtica')
else:
    print('Distribución Mesocúrtica')
```

Asimetría -1.1302
 Curtosis 2.2858
 Distribución Leptocúrtica

```
[14]: # Cargar el set de datos de Iris
from sklearn import datasets
from pprint import pprint

iris = datasets.load_iris()
pprint(iris)

X = iris.data
y = iris.target

# ¿Qué tipo de asimetría se observa en la variable "longitud del pétalo"?
# ???

# ¿A qué tipo de distribución se ajusta la variable "ancho del sépalo", según
# su curtosis?
# ???
```

```
{'DESCR': '.. _iris_dataset:\n'
'\n'
'Iris plants dataset\n'
'-----\n'
'\n'
'**Data Set Characteristics:**\n'
'\n'
':Number of Instances: 150 (50 in each of three classes)\n'
':Number of Attributes: 4 numeric, predictive attributes and the '
'class\n'
':Attribute Information:\n'
'  - sepal length in cm\n'
'  - sepal width in cm\n'
'  - petal length in cm\n'
'  - petal width in cm\n'
'  - class:\n'
'      - Iris-Setosa\n'
'      - Iris-Versicolour\n'
'      - Iris-Virginica\n'
'\n'
':Summary Statistics:\n'
'\n'
'===== \n'
'      Min  Max   Mean    SD   Class Correlation\n'
'===== \n'
'sepal length:  4.3  7.9   5.84   0.83    0.7826\n'
'sepal width:   2.0  4.4   3.05   0.43   -0.4194\n'
```



```

'petal length:  1.0  6.9  3.76  1.76  0.9490  (high!)\n'
'petal width:   0.1  2.5  1.20  0.76  0.9565  (high!)\n'
'=====  ====  ====  =====  =====  =====\n'
'\n'
':Missing Attribute Values: None\n'
':Class Distribution: 33.3% for each of 3 classes.\n'
':Creator: R.A. Fisher\n'
':Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n'
':Date: July, 1988\n'
'\n'
'The famous Iris database, first used by Sir R.A. Fisher. The '
'dataset is taken\n'
'from Fisher's paper. Note that it's the same as in R, but not as in "
'the UCI\n'
'Machine Learning Repository, which has two wrong data points.\n'
'\n'
'This is perhaps the best known database to be found in the\n'
'pattern recognition literature. Fisher's paper is a classic in the "
'field and\n'
'is referenced frequently to this day. (See Duda & Hart, for '
'example.) The\n'
'data set contains 3 classes of 50 instances each, where each class '
'refers to a\n'
'type of iris plant. One class is linearly separable from the other '
'2; the\n'
'latter are NOT linearly separable from each other.\n'
'\n'
'.. dropdown:: References\n'
'\n'
' - Fisher, R.A. "The use of multiple measurements in taxonomic '
'problems"\n'
'   Annual Eugenics, 7, Part II, 179-188 (1936); also in '
'"Contributions to\n'
'   Mathematical Statistics" (John Wiley, NY, 1950).\n'
' - Duda, R.O., & Hart, P.E. (1973) Pattern Classification and '
'Scene Analysis.\n'
'   (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page '
'218.\n'
' - Dasarthy, B.V. (1980) "Nosing Around the Neighborhood: A New '
'System\n'
'   Structure and Classification Rule for Recognition in Partially '
'Exposed\n'
'   Environments". IEEE Transactions on Pattern Analysis and '
'Machine\n'
'   Intelligence, Vol. PAMI-2, No. 1, 67-71.\n'
' - Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE '
'Transactions\n'
'   on Information Theory, May 1972, 431-433.\n'

```

```

' - See also: 1988 MLC Proceedings, 54-64. Cheeseman et al"s '
'AUTOCLASS II\n'
' conceptual clustering system finds 3 classes in the data.\n'
' - Many, many more ...\n',
'data': array([[5.1, 3.5, 1.4, 0.2],
[4.9, 3. , 1.4, 0.2],
[4.7, 3.2, 1.3, 0.2],
[4.6, 3.1, 1.5, 0.2],
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
[5.4, 3.9, 1.3, 0.4],
[5.1, 3.5, 1.4, 0.3],
[5.7, 3.8, 1.7, 0.3],
[5.1, 3.8, 1.5, 0.3],
[5.4, 3.4, 1.7, 0.2],
[5.1, 3.7, 1.5, 0.4],
[4.6, 3.6, 1. , 0.2],
[5.1, 3.3, 1.7, 0.5],
[4.8, 3.4, 1.9, 0.2],
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],

```

[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1.],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1.],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1.],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1.],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1.],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1.],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],

[5.8, 2.6, 4. , 1.2],
 [5. , 2.3, 3.3, 1.],
 [5.6, 2.7, 4.2, 1.3],
 [5.7, 3. , 4.2, 1.2],
 [5.7, 2.9, 4.2, 1.3],
 [6.2, 2.9, 4.3, 1.3],
 [5.1, 2.5, 3. , 1.1],
 [5.7, 2.8, 4.1, 1.3],
 [6.3, 3.3, 6. , 2.5],
 [5.8, 2.7, 5.1, 1.9],
 [7.1, 3. , 5.9, 2.1],
 [6.3, 2.9, 5.6, 1.8],
 [6.5, 3. , 5.8, 2.2],
 [7.6, 3. , 6.6, 2.1],
 [4.9, 2.5, 4.5, 1.7],
 [7.3, 2.9, 6.3, 1.8],
 [6.7, 2.5, 5.8, 1.8],
 [7.2, 3.6, 6.1, 2.5],
 [6.5, 3.2, 5.1, 2.],
 [6.4, 2.7, 5.3, 1.9],
 [6.8, 3. , 5.5, 2.1],
 [5.7, 2.5, 5. , 2.],
 [5.8, 2.8, 5.1, 2.4],
 [6.4, 3.2, 5.3, 2.3],
 [6.5, 3. , 5.5, 1.8],
 [7.7, 3.8, 6.7, 2.2],
 [7.7, 2.6, 6.9, 2.3],
 [6. , 2.2, 5. , 1.5],
 [6.9, 3.2, 5.7, 2.3],
 [5.6, 2.8, 4.9, 2.],
 [7.7, 2.8, 6.7, 2.],
 [6.3, 2.7, 4.9, 1.8],
 [6.7, 3.3, 5.7, 2.1],
 [7.2, 3.2, 6. , 1.8],
 [6.2, 2.8, 4.8, 1.8],
 [6.1, 3. , 4.9, 1.8],
 [6.4, 2.8, 5.6, 2.1],
 [7.2, 3. , 5.8, 1.6],
 [7.4, 2.8, 6.1, 1.9],
 [7.9, 3.8, 6.4, 2.],
 [6.4, 2.8, 5.6, 2.2],
 [6.3, 2.8, 5.1, 1.5],
 [6.1, 2.6, 5.6, 1.4],
 [7.7, 3. , 6.1, 2.3],
 [6.3, 3.4, 5.6, 2.4],
 [6.4, 3.1, 5.5, 1.8],
 [6. , 3. , 4.8, 1.8],
 [6.9, 3.1, 5.4, 2.1],

```

        [6.7, 3.1, 5.6, 2.4],
        [6.9, 3.1, 5.1, 2.3],
        [5.8, 2.7, 5.1, 1.9],
        [6.8, 3.2, 5.9, 2.3],
        [6.7, 3.3, 5.7, 2.5],
        [6.7, 3. , 5.2, 2.3],
        [6.3, 2.5, 5. , 1.9],
        [6.5, 3. , 5.2, 2. ],
        [6.2, 3.4, 5.4, 2.3],
        [5.9, 3. , 5.1, 1.8]]),
'data_module': 'sklearn.datasets.data',
'feature_names': ['sepal length (cm)',
                  'sepal width (cm)',
                  'petal length (cm)',
                  'petal width (cm)'],
'filename': 'iris.csv',
'frame': None,
'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10')}
```

```

[15]: # EJERCICIO EXTRA SOBRE EL DATASET DE IRIS
      # Extraer la media y la desviación típica de cada variable en función de la
      ↪ clase y representarlo en un gráfico (plt.errorbar)
```

```
[ ]:
```

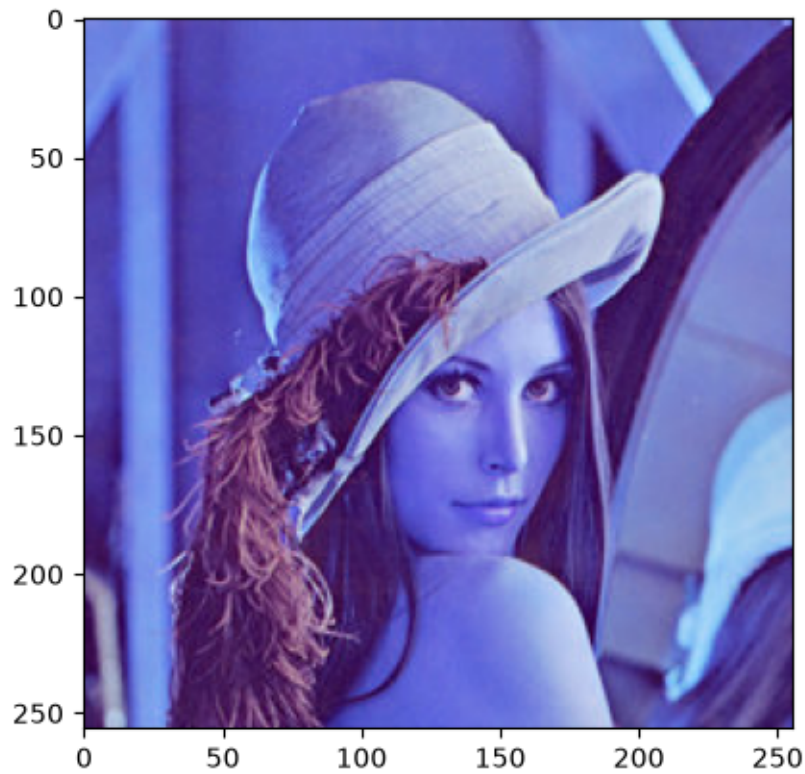
3 3_2_2_transformaciones

```

[19]: # READ AN IMAGE
      # pip install opencv-python
      import matplotlib.pyplot as plt
      import cv2
      import numpy as np

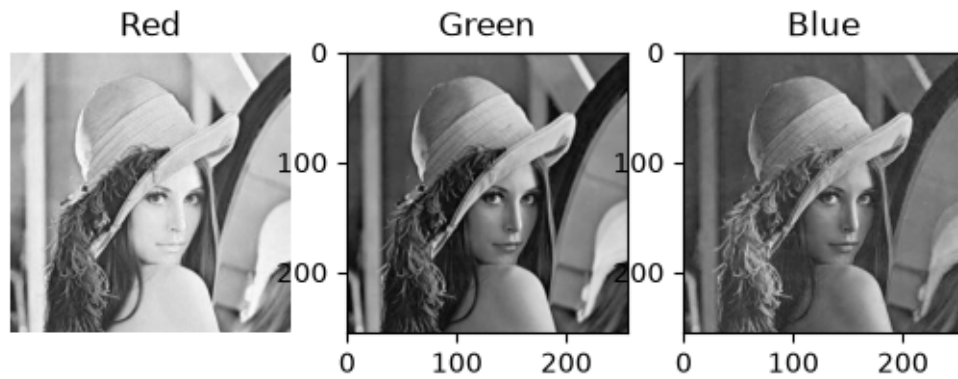
      img = cv2.imread('images/Lena_RGB.png')
      print('Image dimensions: ', np.shape(img))
      plt.imshow(img, cmap='gray')
      plt.show()
```

Image dimensions: (256, 256, 3)

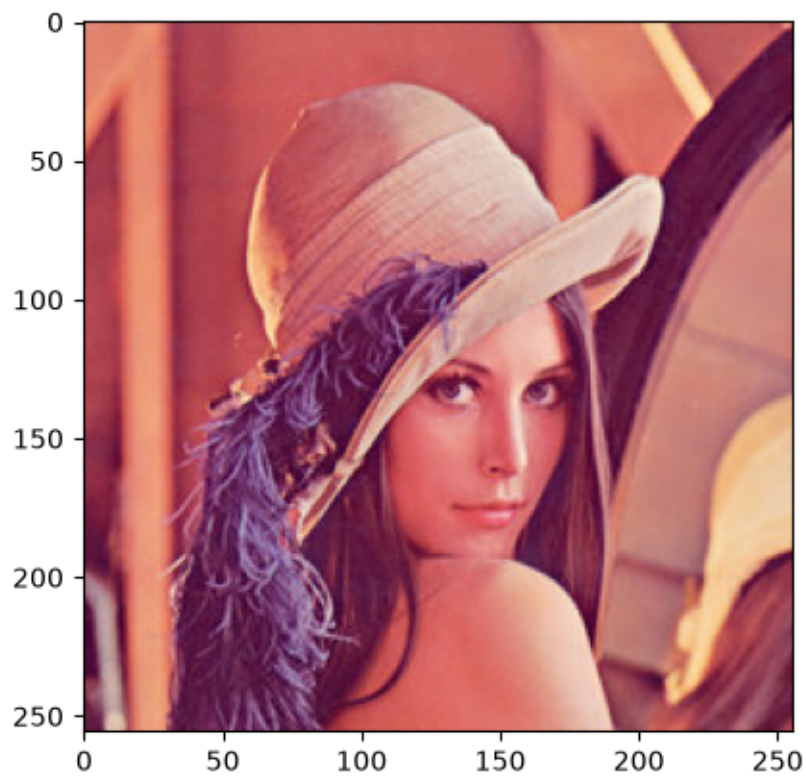


```
[20]: # Extraer por separado la imagen de grises de cada canal
R = img[:, :, 2]
G = img[:, :, 1]
B = img[:, :, 0]
```

```
[21]: # Visualizar los canales en un subplot
fig, ax = plt.subplots(1, 3)
ax[0].imshow(R, cmap='gray', ax[0].set_title('Red'), ax[0].axis('off')
ax[1].imshow(G, cmap='gray', ax[1].set_title('Green')
ax[2].imshow(B, cmap='gray', ax[2].set_title('Blue')
plt.show()
```



```
[22]: # Convertir BGR en RGB
RGB_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(RGB_img, cmap='gray')
plt.show()
```



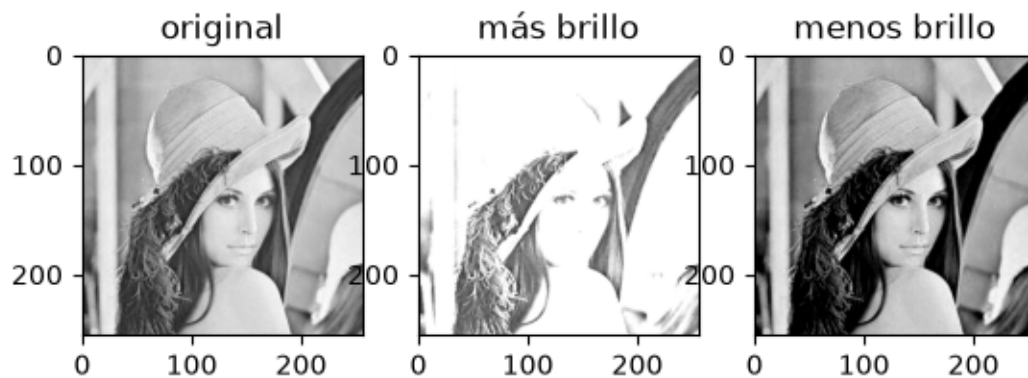
3.1 TRANSFORMACIONES DE INTENSIDAD

```
[25]: # CAMBIO DE BRILLO
img = cv2.imread('images/Lena_RGB.png')
img = img[:, :, 2] # red color

mas_brillo = 100
menos_brillo = -100

mas_brillo_img = cv2.add(img, mas_brillo) # Importante el "cv2.add" en vez de "+"
menos_brillo_img = cv2.add(img, menos_brillo)

fig, ax = plt.subplots(1, 3)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(mas_brillo_img, cmap='gray'), ax[1].set_title('más brillo')
ax[2].imshow(menos_brillo_img, cmap='gray'), ax[2].set_title('menos brillo')
plt.show()
```



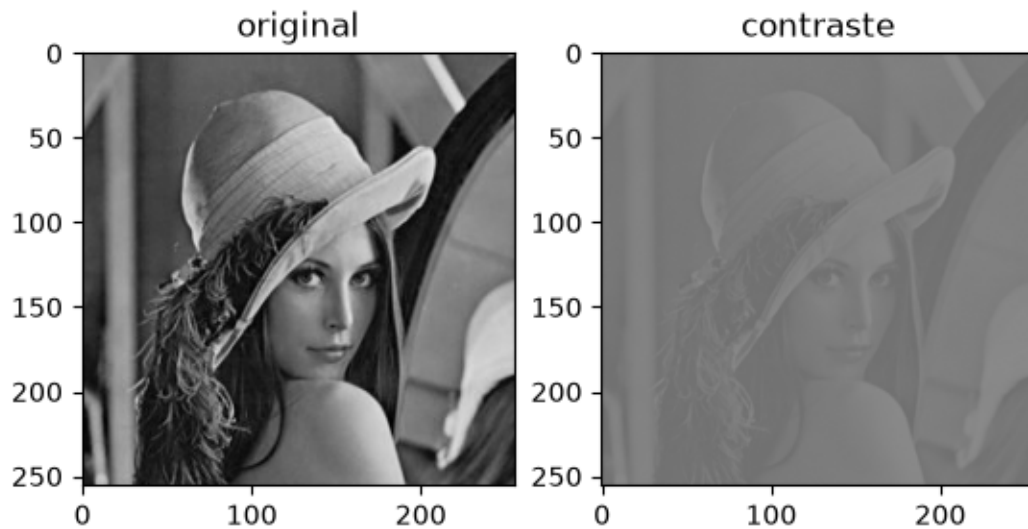
```
[27]: # CAMBIO DE CONTRASTE de acuerdo con el programa GIMP
img = cv2.imread('images/Lena_RGB.png')
img = img[:, :, 1] # canal verde

contraste = -100
f = 131*(contraste + 127)/(127*(131-contraste))
alpha_c = f
gamma_c = 127*(1-f)
contrast_img = cv2.addWeighted(img, alpha_c, img, 0, gamma_c)

fig, ax = plt.subplots(1, 2)
ax[0].imshow(img, cmap='gray', vmin=0, vmax=255), ax[0].set_title('original')
ax[1].imshow(contrast_img, cmap='gray', vmin=0, vmax=255), ax[1].
    set_title('contraste')
```



```
plt.show()
```



3.2 CONVERSIONES DEL ESPACIO DE COLOR

```
[40]: # RGB to CMYK
import numpy as np
from skimage import io

img = io.imread('images/colores.png')
rgb = img.copy()
rgb_p = rgb.astype('uint8')/255

with np.errstate(invalid='ignore', divide='ignore'):
    K = 1 - np.max(rgb_p, axis=2)
    # Extrae los canales de acuerdo con la ecuación del power point
    C = ???
    M = ???
    Y = ???

CMYK = (np.dstack((C,M,Y,K))*255).astype('uint8')
C,M,Y,K = cv2.split(CMYK)

fig, ax = plt.subplots(2,3)
ax[0,0].imshow(img, cmap='gray'), ax[0,0].set_title('original')
ax[0,1].imshow(CMYK.astype('uint8'), cmap='gray'), ax[0,1].set_title('CMYK')
ax[0,2].imshow(C.astype('uint8'), cmap='gray'), ax[0,2].set_title('cyan')
ax[1,0].imshow(M.astype('uint8'), cmap='gray'), ax[1,0].set_xlabel('magenta')
ax[1,1].imshow(Y.astype('uint8'), cmap='gray'), ax[1,1].set_xlabel('yellow')
```

```
ax[1,2].imshow(K.astype('uint8'), cmap='gray'), ax[1,2].set_xlabel('black')
plt.show()
```

Cell In[40], line 12

```
C = ???
```

SyntaxError: invalid syntax

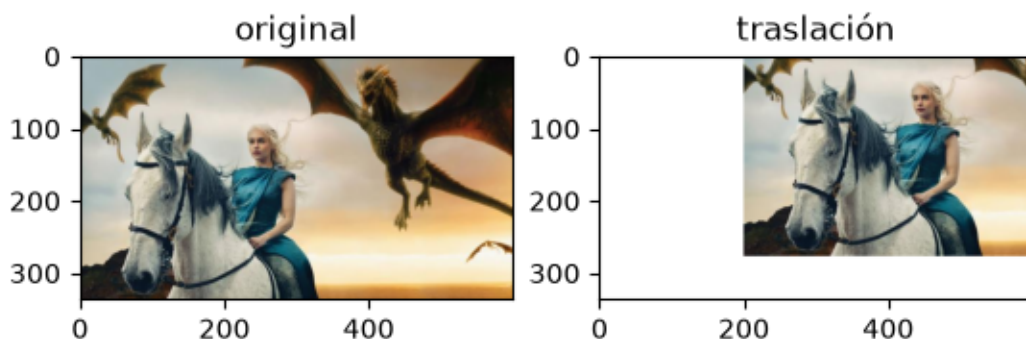
```
[41]: # Otras conversiones
from skimage import io

img = cv2.imread('images/Lena_RGB.png')
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # gray-scale
HSV = cv2.cvtColor(img, cv2.COLOR_BGR2HSV) # (H)ue, (S)aturation and (V)alue
Lab = cv2.cvtColor(img, cv2.COLOR_BGR2Lab) # (L)uminosidad, a-b
YCrCb = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb) # Y-Luma, Cr-Cb crominancia
rojo y azul
```

3.3 TRANSFORMACIONES GEOMÉTRICAS

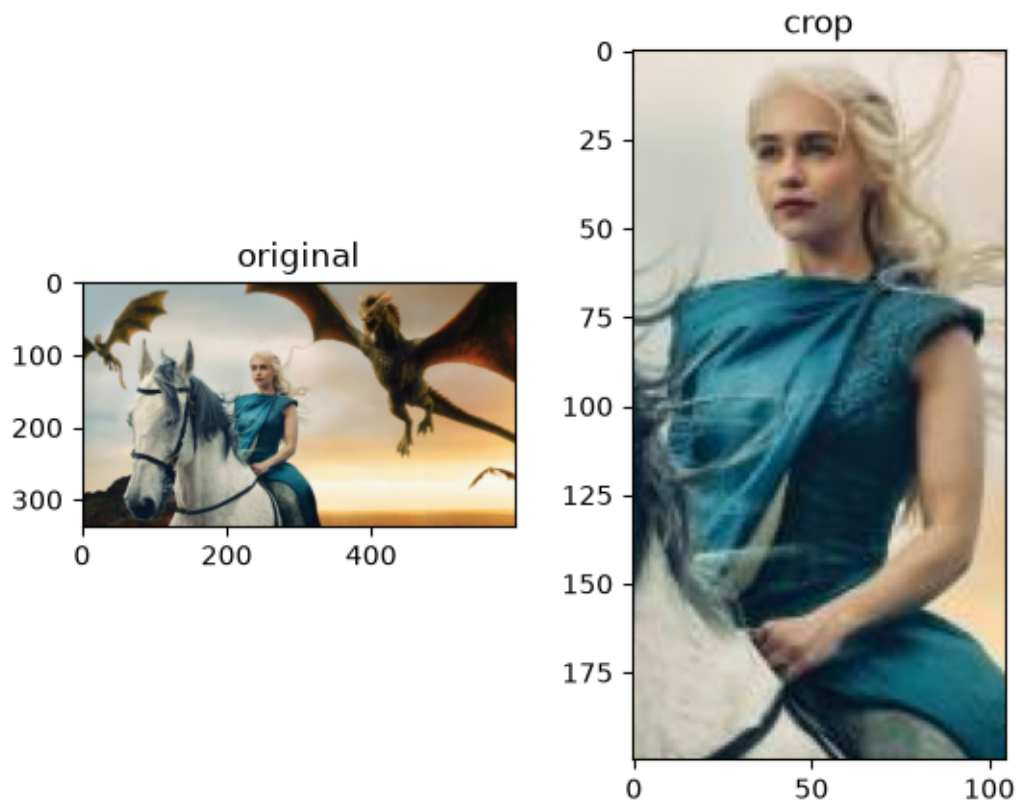
```
[42]: # TRASLACIÓN
img = io.imread('images/GOT.png')
rows, cols, ch = img.shape
M = np.float32([[1,0,200],[0,1,-60]]) # Defino la matriz de transformación
new_img = cv2.warpAffine(img,M,(cols,rows)) # Aplico la transformación

figs, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(new_img, cmap='gray'), ax[1].set_title('traslación')
plt.show()
```



```
[46]: # CROPPING
img = io.imread('images/GOT.png')
new_img = img[90:290, 200:305]

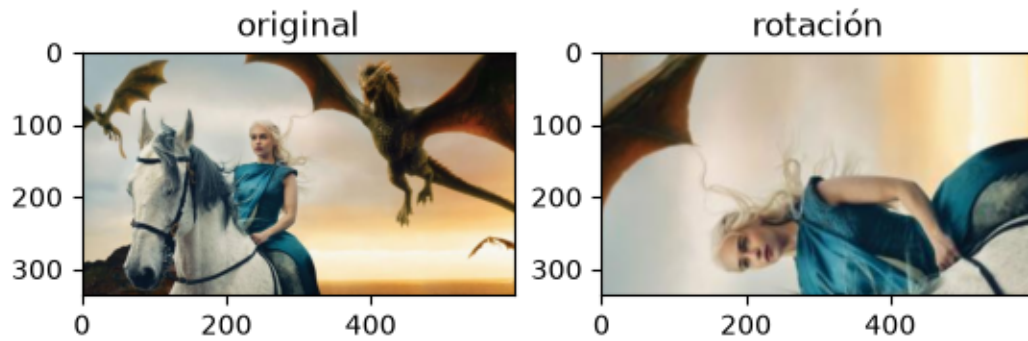
figs, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(new_img, cmap='gray'), ax[1].set_title('crop')
plt.show()
```



```
[47]: # ROTACIÓN
img = io.imread('images/GOT.png')
rows, cols, ch = img.shape
M = cv2.getRotationMatrix2D((cols/2,rows/2),angle=90,scale=2) # Defino la
    ↪matriz de transformación
new_img = cv2.warpAffine(img,M,(cols,rows)) # Aplico la transformación

figs, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(new_img, cmap='gray'), ax[1].set_title('rotación')
```

```
plt.show()
```



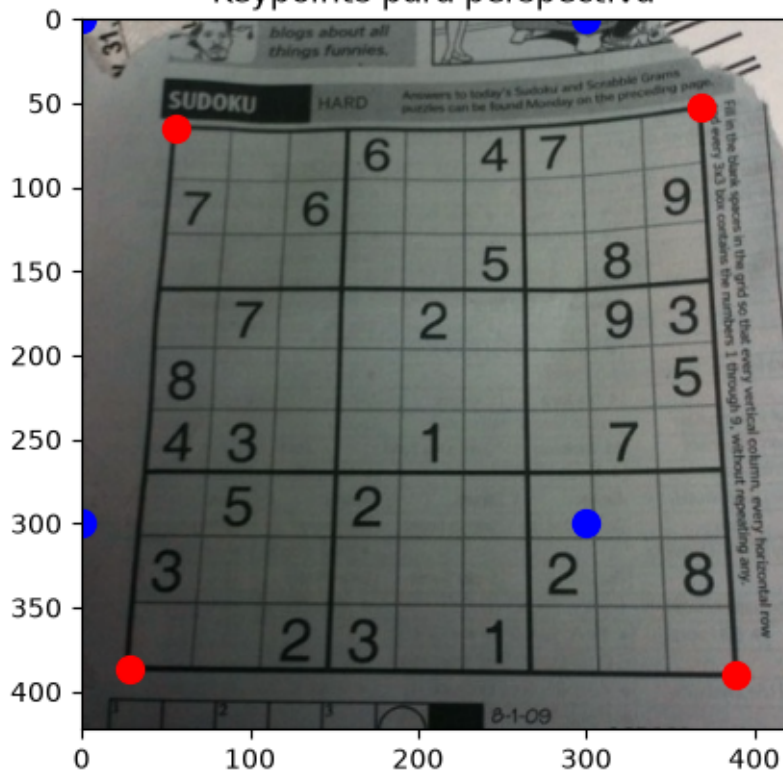
```
[48]: # PERSPECTIVA
img = cv2.imread('images/sudoku.png')
rows, cols, ch = img.shape
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])

plt.imshow(img, cmap='gray')
for i in range(0,4):
    plt.plot(pts1[i,0], pts1[i,1], 'or', markersize=10)
    plt.plot(pts2[i,0], pts2[i,1], 'ob', markersize=10)
plt.title('Keypoints para perspectiva')
plt.show()

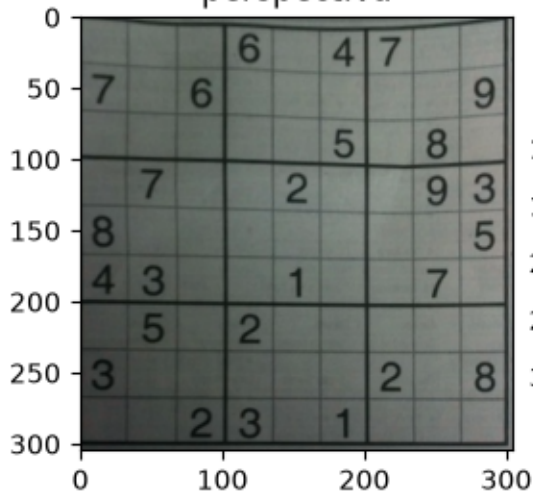
M = cv2.getPerspectiveTransform(pts1,pts2) # Defino la matriz de transformación
pers = cv2.warpPerspective(img,M,(305,305)) # Aplico la transformación
crop = img[50:400,20:400]

figs, ax = plt.subplots(1,2)
ax[0].imshow(pers, cmap='gray'), ax[0].set_title('perspectiva')
ax[1].imshow(crop, cmap='gray'), ax[1].set_title('crop')
plt.show()
```

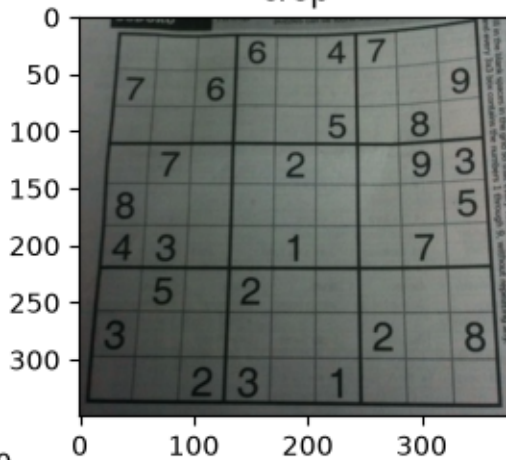
Keypoints para perspectiva



perspectiva



crop

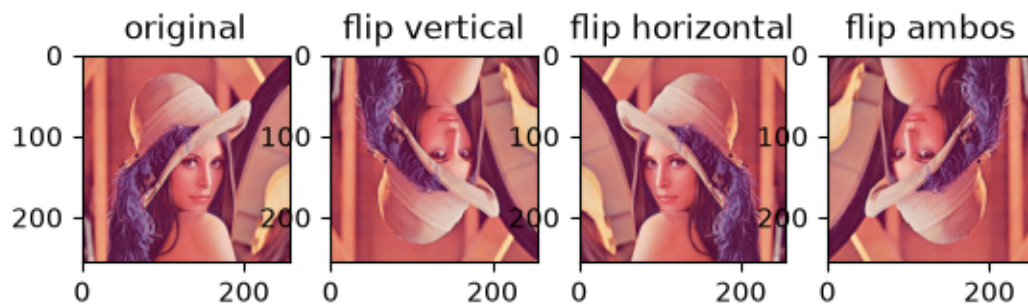


```
[49]: # FLIPPING
      # Leer la imagen "Lena_RGB.png" en formato RGB
```

```
img = io.imread('images/Lena_RGB.png')

# Voltear la imagen para conseguir las siguientes transformaciones. Utiliza el
# método "cv2.flip()"
flipVertical = cv2.flip(img,0)
flipHorizontal = cv2.flip(img,1)
flipBoth = cv2.flip(img,-1)

figs, ax = plt.subplots(1,4)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(flipVertical, cmap='gray'), ax[1].set_title('flip vertical')
ax[2].imshow(flipHorizontal, cmap='gray'), ax[2].set_title('flip horizontal')
ax[3].imshow(flipBoth, cmap='gray'), ax[3].set_title('flip ambos')
plt.show()
```



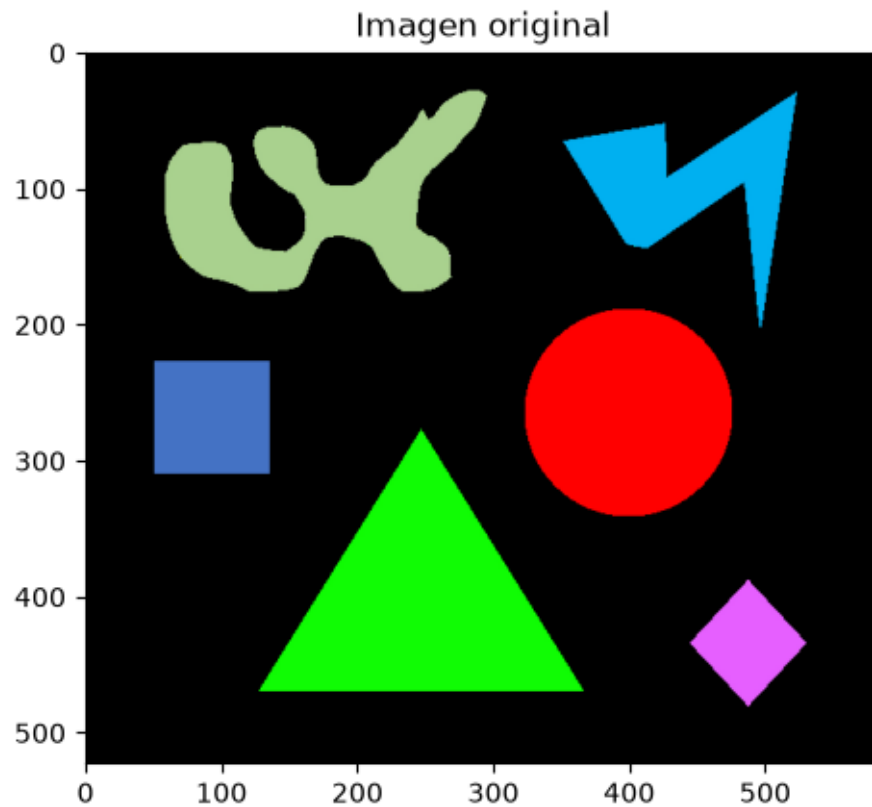
[]:

4 3_2_3_Image_segmentation

4.1 SEGMENTACIÓN MEDIANTE THRESHOLDING

```
[50]: import matplotlib.pyplot as plt
import cv2

# Read an image
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```



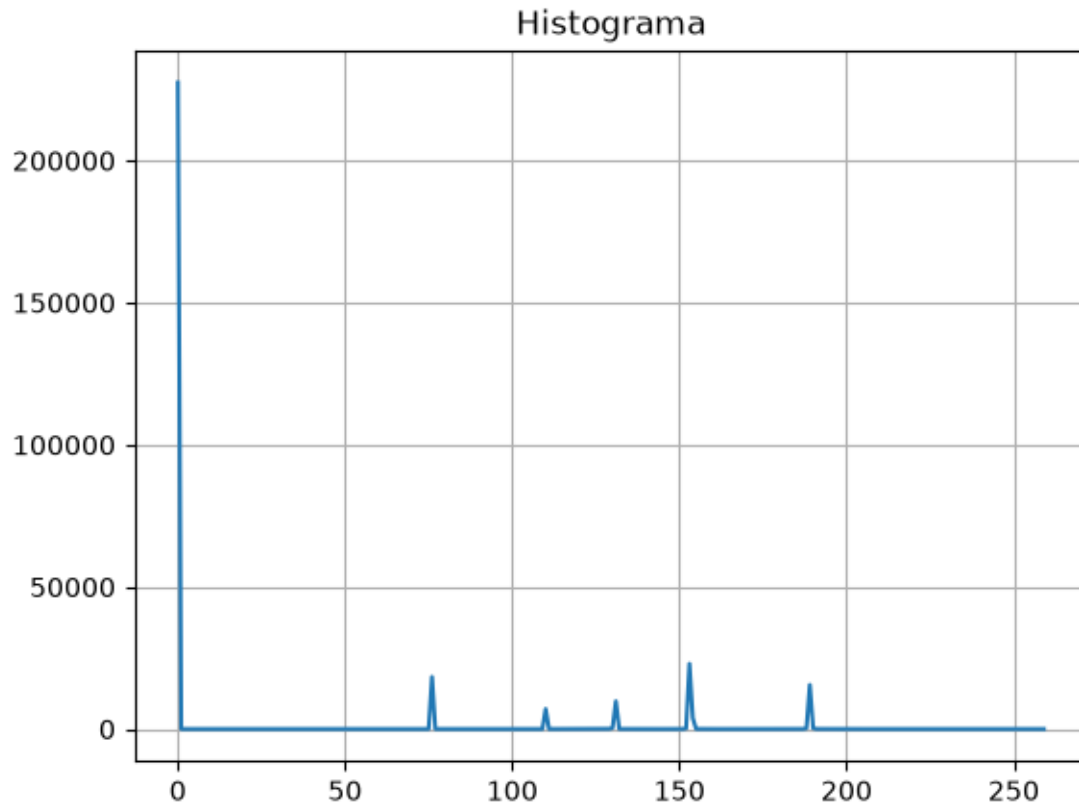
```
[51]: # Convertimos la imagen a escala de grises
import numpy as np

gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# gray_img = cv2.imread('figuras.png', 0) # Una forma de leer la imagen
# directamente en escala de grises
plt.imshow(gray_img, cmap='gray')
plt.title('Imagen de grises')
plt.show()
```



```
[52]: # Calculamos el histograma
hist = cv2.calcHist([gray_img], [0], None, [260], [0, 260]) # [0] son los
    ↪ canales, y es el canal gris None la máscara
plt.plot(hist)
plt.grid()
plt.title('Histograma')
plt.show()

# Otra forma de mostrar el histograma (solo visualización)
# plt.hist(gray_img.ravel(), bins=50)
# plt.grid(True)
# plt.show()
```

```
[53]: # Fijamos el umbral en base al histograma anterior  
t = 50
```

```
[54]: # Extraemos la máscara binaria  
maxim = 255  
_, mask = cv2.threshold(gray_img, t, maxim, cv2.THRESH_BINARY)  
  
# Otra formas de extraer la máscara  
# mask = gray_img.copy()  
# mask = mask>t  
  
# Visualizamos para corroborar  
plt.imshow(mask, cmap='gray')  
plt.title('Máscara t=' + str(t))  
plt.show()  
print(np.unique(mask)) # Atent@s a los formatos (bool, uint8, etc.)
```

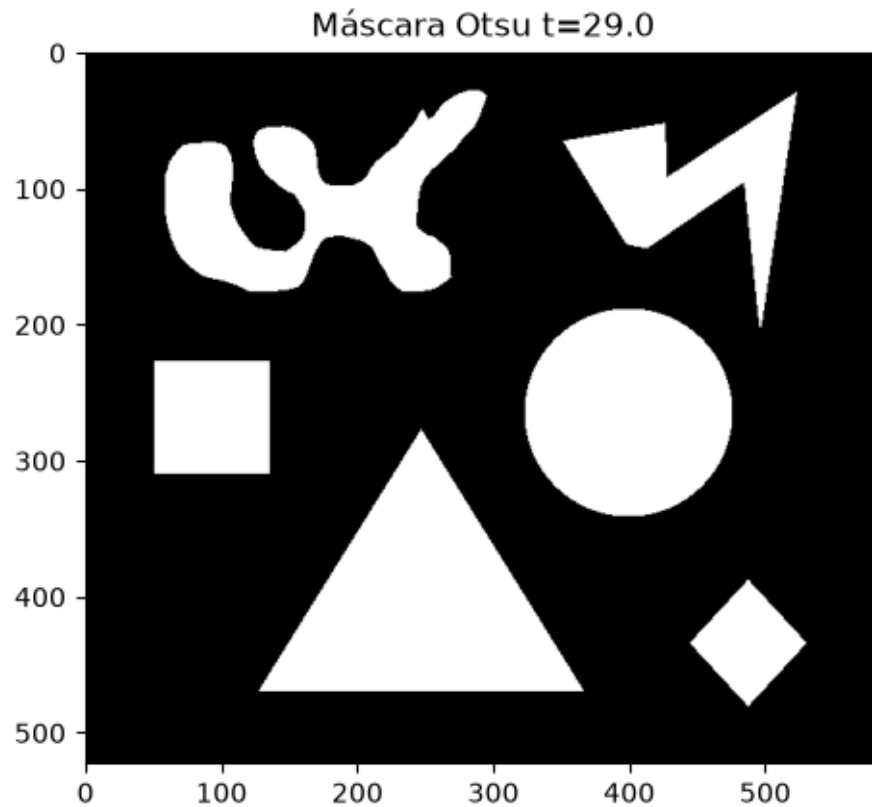


[0 255]

```
[55]: # Fijamos el umbral con el método de OTSU
t, mask = cv2.threshold(gray_img,0,1,cv2.THRESH_OTSU) # 0 es por defecto y 1
↳ es el valor máximo de la máscara
print(np.unique(mask))

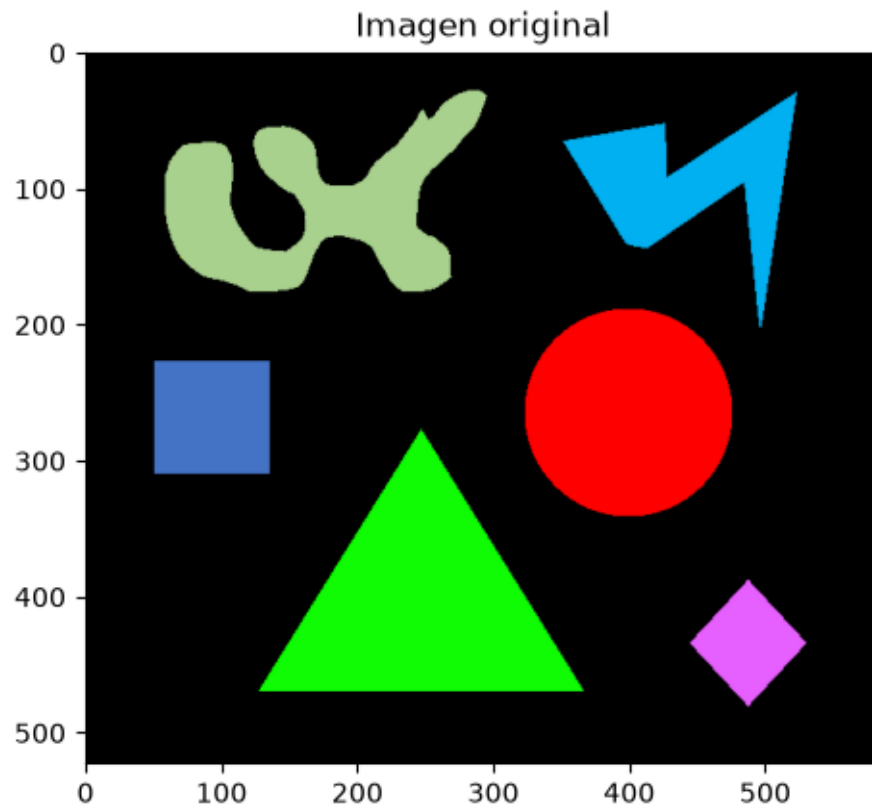
# Visualizamos para corroborar que se obtiene el mismo resultado
plt.imshow(mask, cmap='gray')
plt.title('Máscara Otsu t=' + str(t))
plt.show()
```

[0 1]

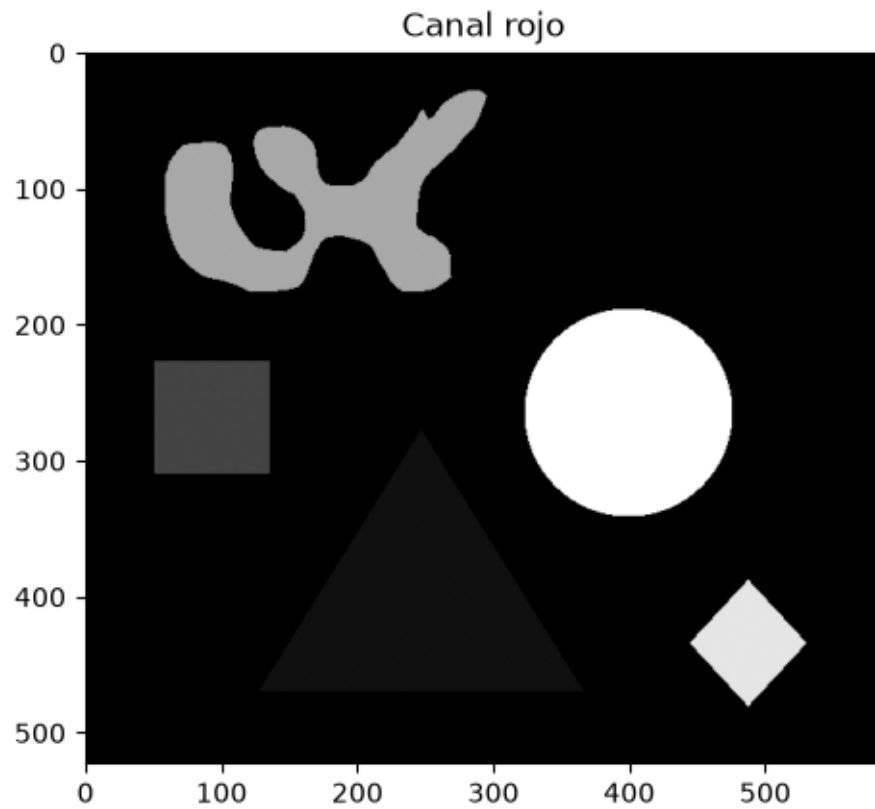


4.2 SEGMENTACIÓN MEDIANTE CONVERSIONES DE COLOR

```
[57]: # Imaginamos que queremos segmentar únicamente el círculo
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```

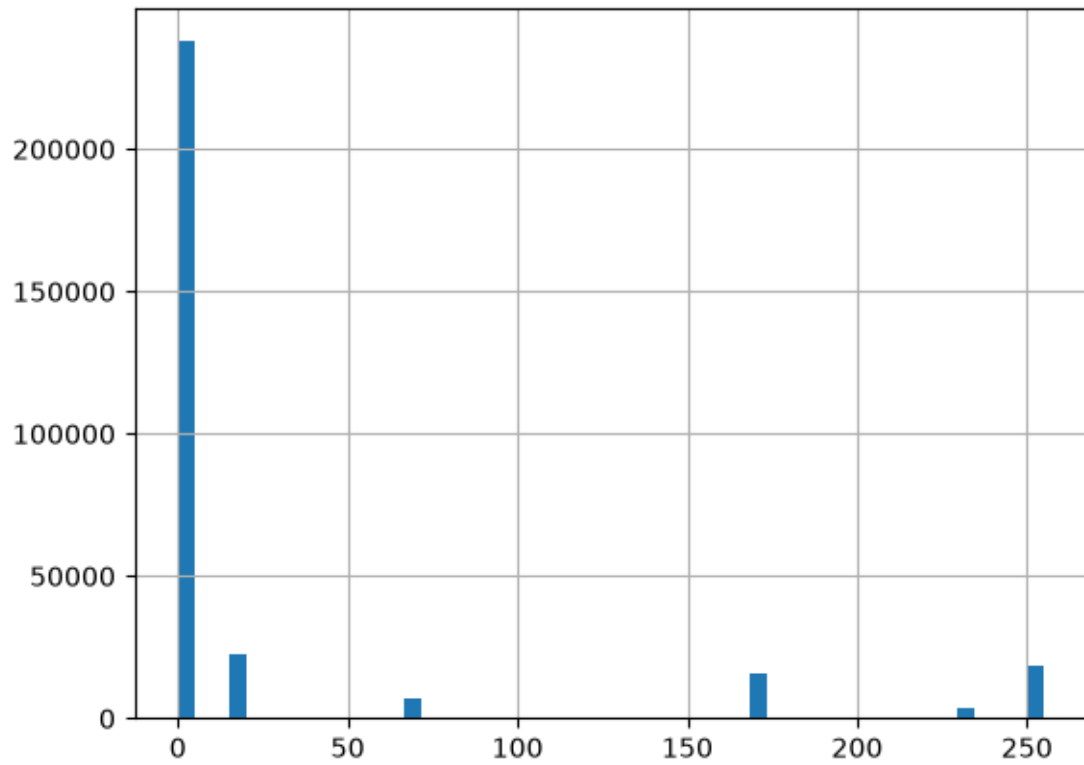


```
[58]: # Obtenemos el canal rojo en bgr la posición 2 es el color rojo
R = img[:, :, 2]
plt.imshow(R, cmap='gray')
plt.title('Canal rojo')
plt.show()
```



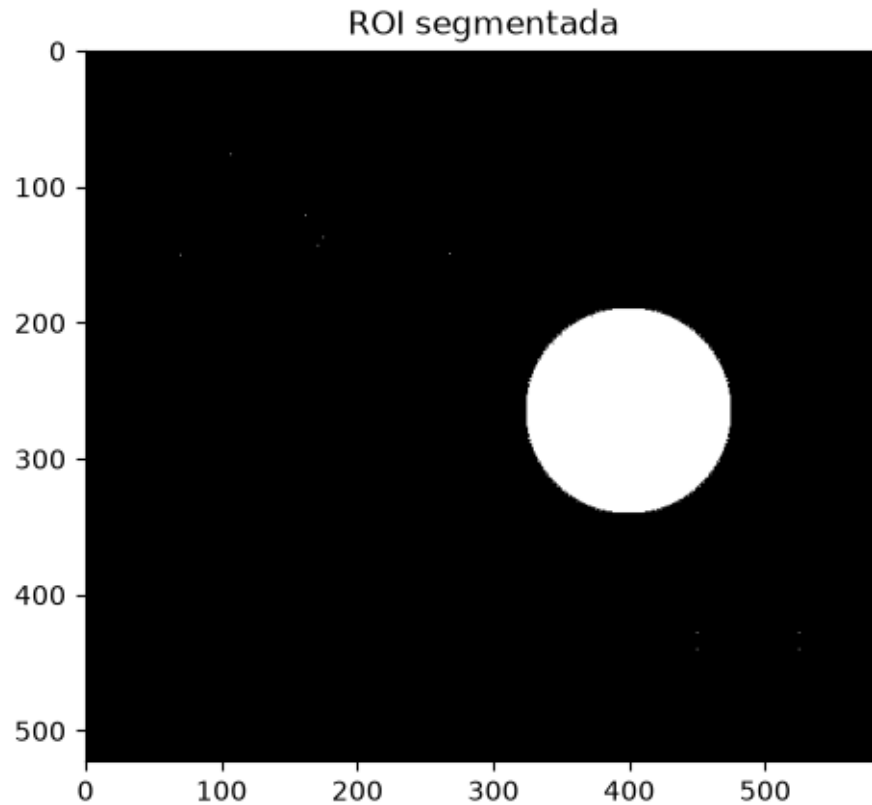
```
[59]: # Observamos el histograma
plt.hist(R.ravel(), bins=50)
plt.grid(True)
plt.show()

print(np.max(R))
```



255

```
[60]: # Fijamos el umbral y extraemos la máscara
t = np.max(R)
mask = R.copy()
mask[mask<t] = 0
plt.imshow(mask, cmap='gray')
plt.title('ROI segmentada')
plt.show()
```



```
[61]: # Ejercicio individual
# Obtener una máscara con la imagen del rayo azul mediante segmentación por
# espacio de color

# Función para extraer los colores secundarios
def convert_to_CMYK(rgb_p):
    with np.errstate(invalid='ignore', divide='ignore'):
        K = 1 - np.max(rgb_p, axis=2)
        C = (1-rgb_p[:, :, 0] - K)/(1-K)
        M = (1-rgb_p[:, :, 1] - K)/(1-K)
        Y = (1-rgb_p[:, :, 2] - K)/(1-K)
        CMYK = (np.dstack((C,M,Y,K))*255).astype('uint8')
    return CMYK

# Leer la imagen
img = cv2.imread('images/figuras.png')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
rgb_p = rgb.astype('uint8')/255

# Extraer los colores secundarios
CMYK = ???
```

```

# Extraer los canales por separado
C,M,Y,K = ???
C = C.astype('uint8')

# Calcular la máscara de la figura del rayo
???

# Visualizar la máscara
plt.imshow(C, cmap='gray')
plt.title('ROI segmentada')
plt.show()

```

```

Cell In[61], line 20
    CMYK = ???
    ~
SyntaxError: invalid syntax

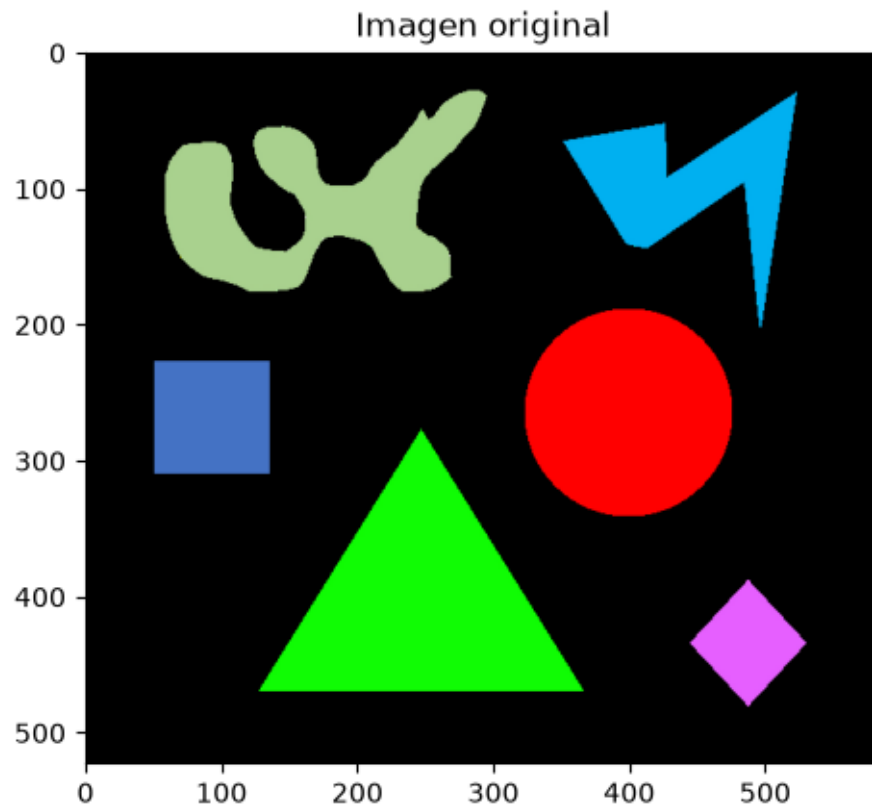
```

4.3 SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES

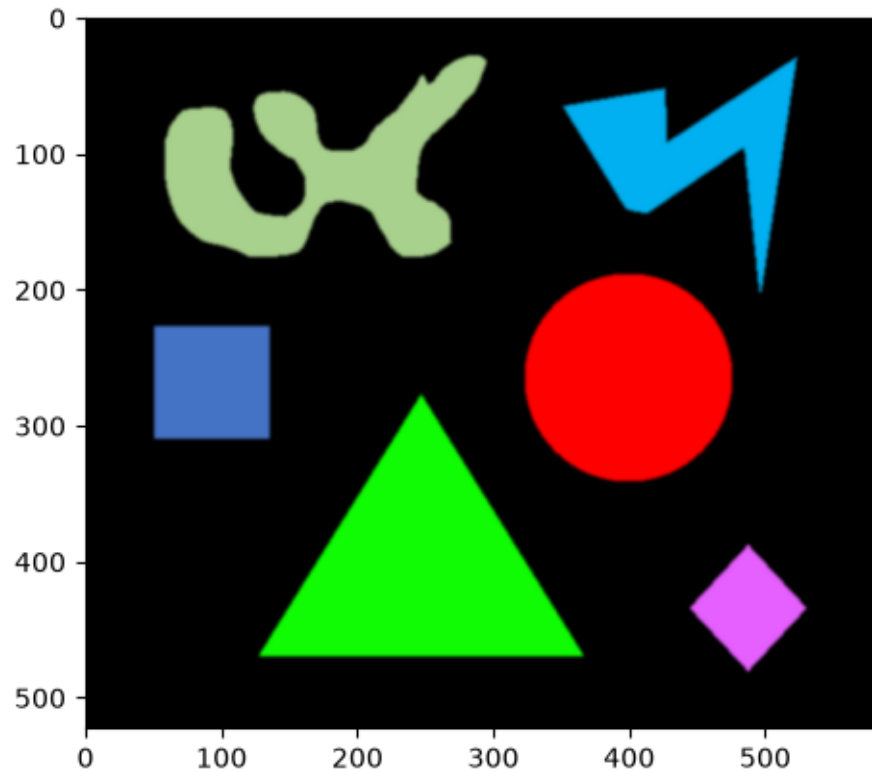
```

[63]: # SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title('Imagen original')
plt.show()

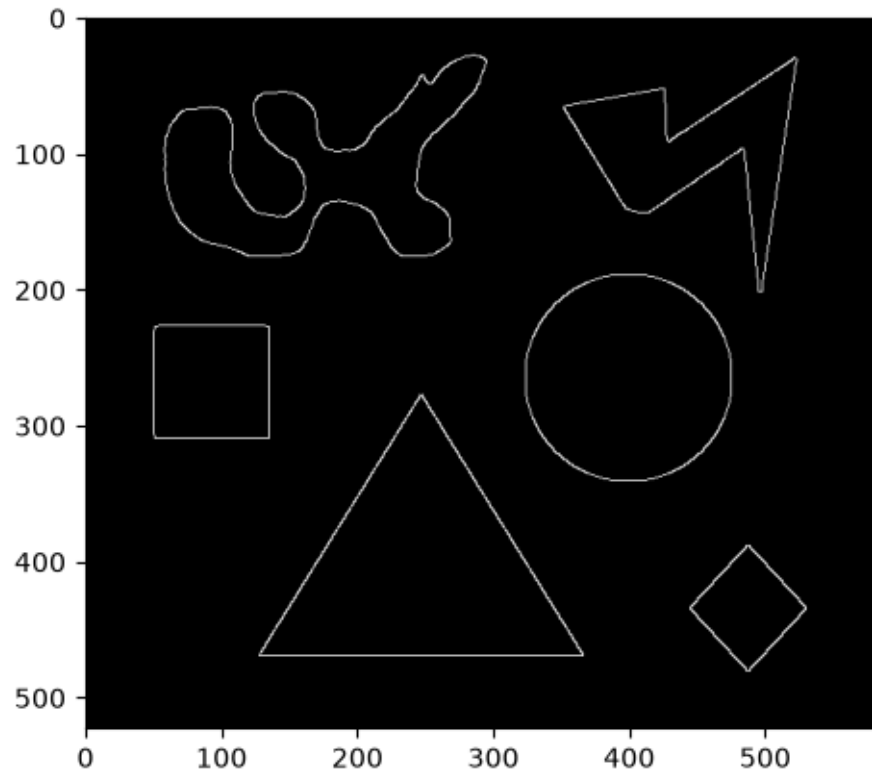
```

```
[64]: # Aplicamos un filtro gaussiano para emborronar las altas frecuencias
img_gaus = cv2.GaussianBlur(img_rgb, (5,5), 0) # (5x5) es el tamaño del filtro
        ↳ y 0 es la desviación estándar
plt.imshow(img_gaus, cmap='gray')
plt.show()
```

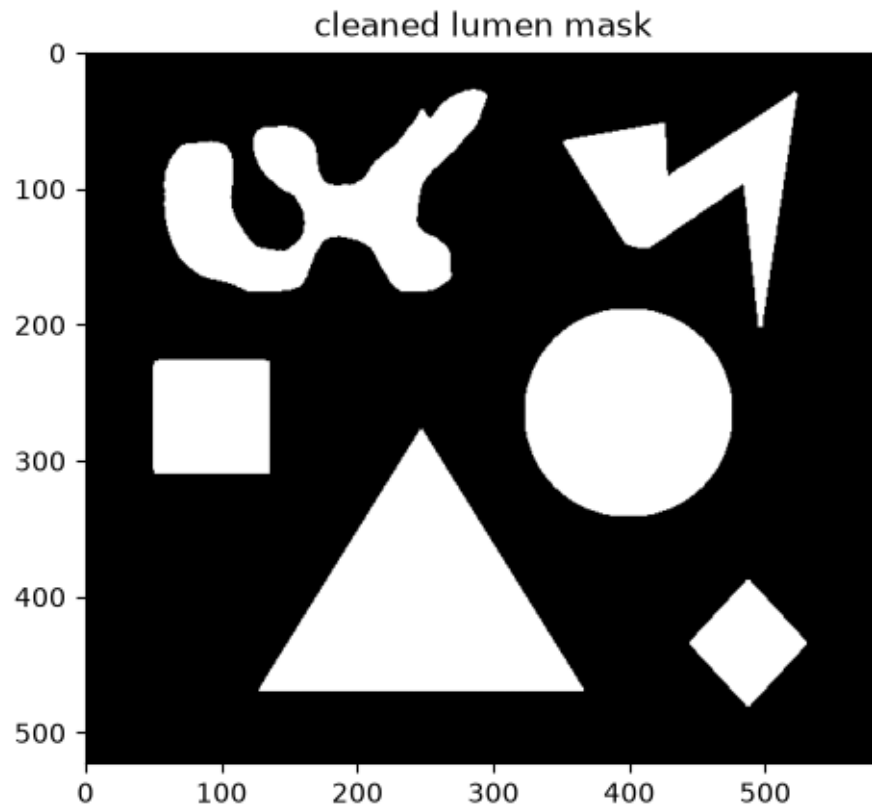


```
[65]: # Detección de bordes de Canny
canny = cv2.Canny(img_gaus, 255/3, 255) # minValue and maxValue. Canny
      ↪ recomienda que minValue sea 3 veces mayor que maxValue
kernel = np.ones((2,2), 'uint8')
# canny = cv2.dilate(canny, kernel, iterations=2)
# canny = cv2.erode(canny, kernel, iterations=2)
plt.imshow(canny, cmap='gray')
plt.show()
```



```
[66]: # Rellenar objetos (solo rellena si no están en contacto con el límite de la
      ↪ imagen)
      from scipy.ndimage import binary_fill_holes as bfh

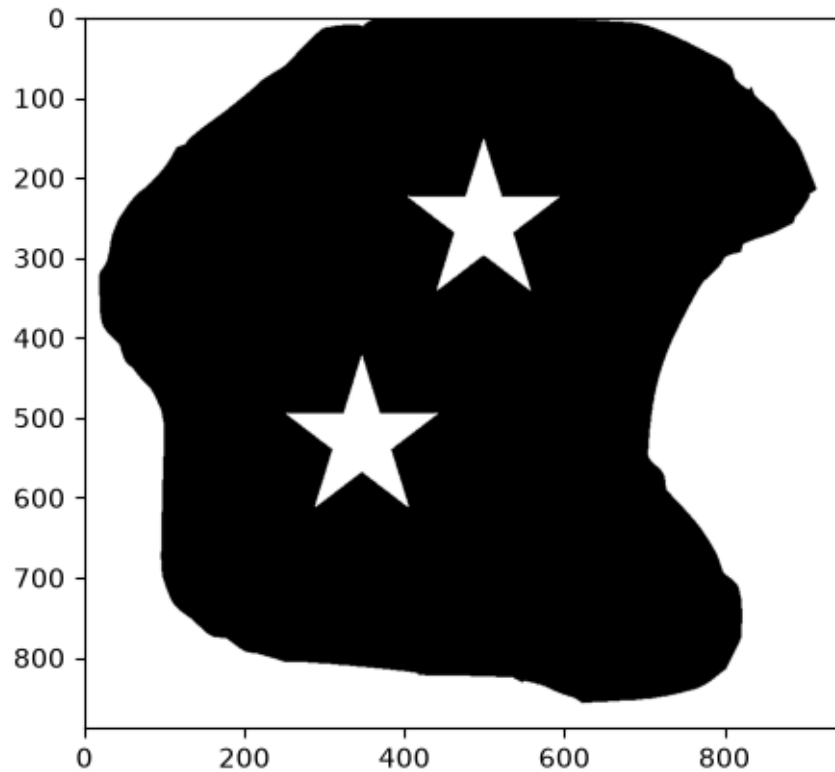
      bordes = canny.copy()
      filled_bordes = bfh(bordes)
      plt.imshow(filled_bordes.astype('uint8'), cmap='gray')
      plt.title('cleaned lumen mask')
      plt.show()
```



4.4 SEGMENTACIÓN POR CRECIMIENTO DE REGIONES (region growing)

```
[68]: # Cargar imagen
from skimage import io
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('images/estrellas.png', 0)
plt.imshow(img, cmap='gray')
plt.show()
```

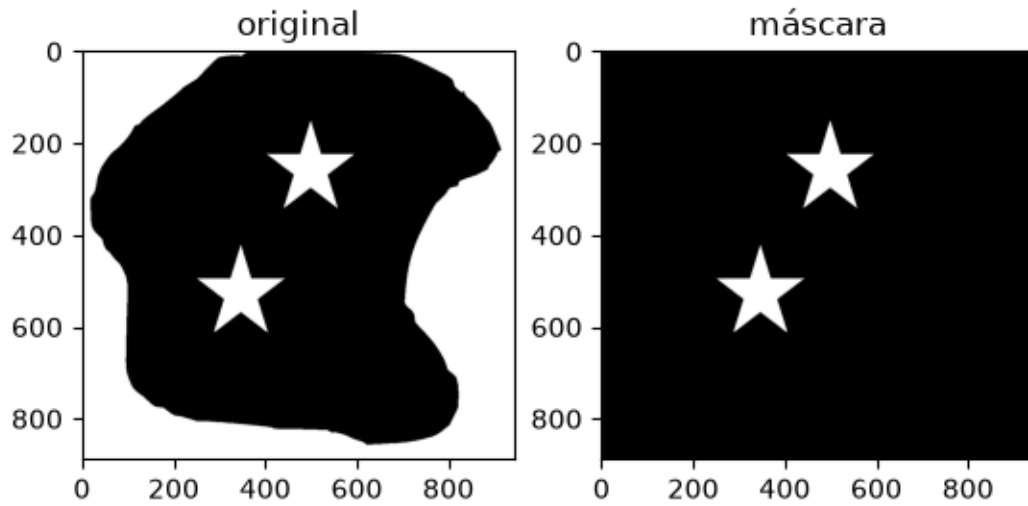


```
[69]: # Crear una copia de la imagen
mask = img.copy()

# Definir una imagen negra de referencia
h, w = img.shape
ref = np.zeros((h+2, w+2), np.uint8)

# Fijar la semilla. En este caso en la esquina superior izquierda (posición
↪ 0,0) para rellenar a partir de ahí
cv2.floodFill(mask, ref, (0,0), 0) # Como nos interesa rellenar de negro,
↪ utilizamos el 0 en la última posición

fig, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(mask, cmap='gray'), ax[1].set_title('máscara')
plt.show()
```

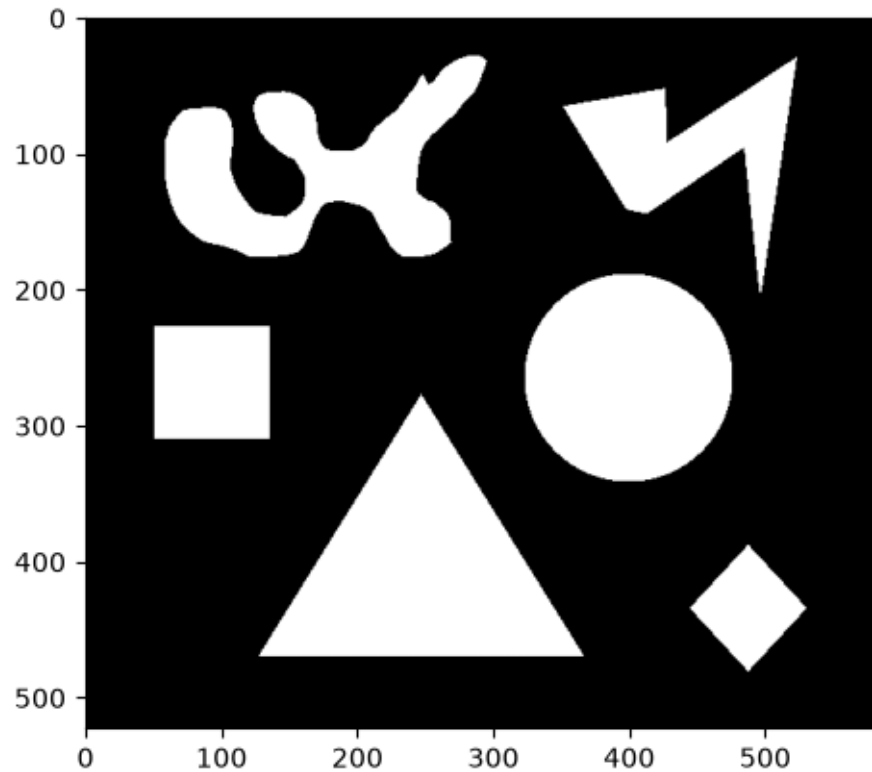


[]:

5 3_2_4_Descriptores_forma

```
[71]: # En primer lugar, obtenemos la máscara de la imagen original mediante la
      ↪ umbralización con Otsu
import matplotlib.pyplot as plt
import cv2
import numpy as np

img = cv2.imread('images/figuras.png')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(img_gray, 0, 1, cv2.THRESH_OTSU)
plt.imshow(mask, cmap='gray')
plt.show()
```



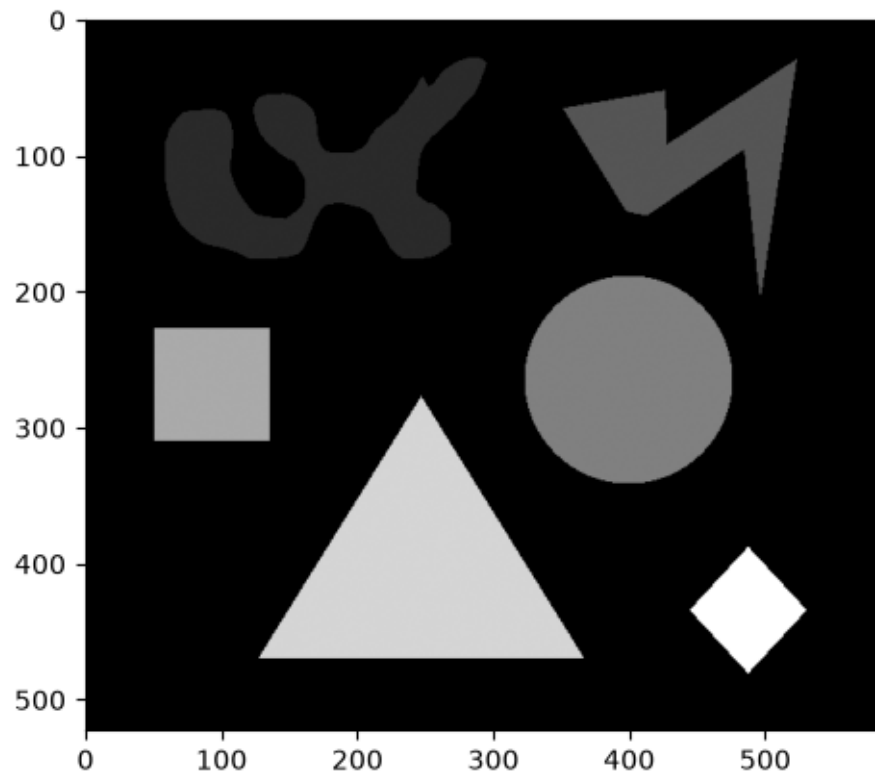
```
[72]: # Convertimos la imagen binaria en una imagen de etiquetas para tener acceso a
      ↪ cada objeto por separado
      from skimage.measure import label

      lab, num = label(mask, return_num=True)

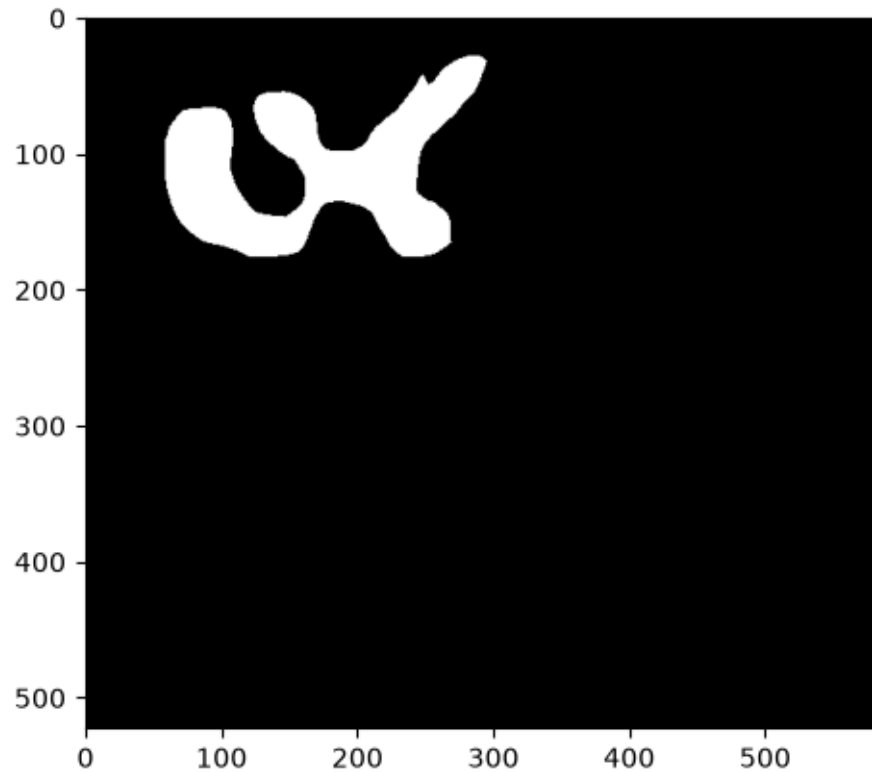
      # Importante:
      # En 'num' devuelve 6 valores correspondientes a los 6 objetos que hay en la
      ↪ imagen.
      # Sin embargo, 'lab' contiene hasta 7 valores diferentes, ya que también tiene
      ↪ en cuenta el fondo de la imagen.
      print('número de objetos: ', num)
      v,c = np.unique(lab, return_counts=True)
      print('posibles valores de intensidad: ', v)
      plt.imshow(lab, cmap='gray')
      plt.show()
```

número de objetos: 6

posibles valores de intensidad: [0 1 2 3 4 5 6]



```
[73]: # Podemos acceder de manera independiente al objeto que nos interese a partir de su etiqueta correspondiente
objeto = lab == 1
objeto = objeto.astype('uint8')
plt.imshow(objeto, cmap='gray')
plt.show()
```

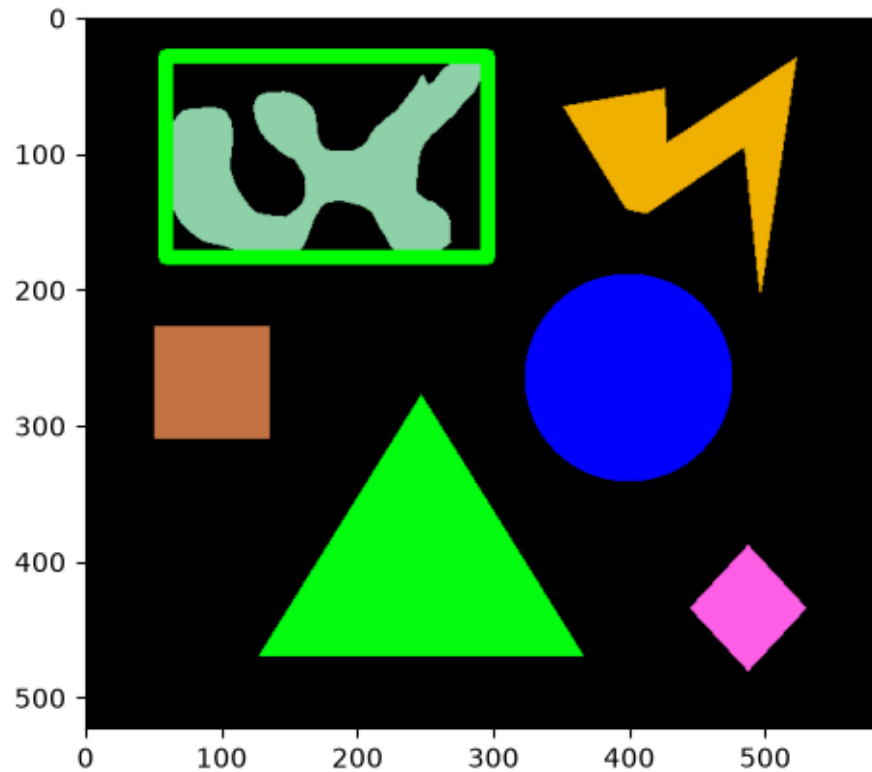



```
[74]: from skimage.measure import regionprops
      # https://scikit-image.org/docs/dev/api/skimage.measure.html#skimage.measure.
      ↪ regionprops

      # Después, podemos extraer la bounding box de dicho objeto
      prop = regionprops(objeto)
      bbox = prop[0].bbox
      print(bbox)

      start_point = (bbox[1], bbox[0])
      end_point = (bbox[3], bbox[2])
      window = cv2.rectangle(img.copy(), start_point, end_point, (0, 255, 0), 10)
      plt.imshow(window, cmap='gray')
      plt.show()
```

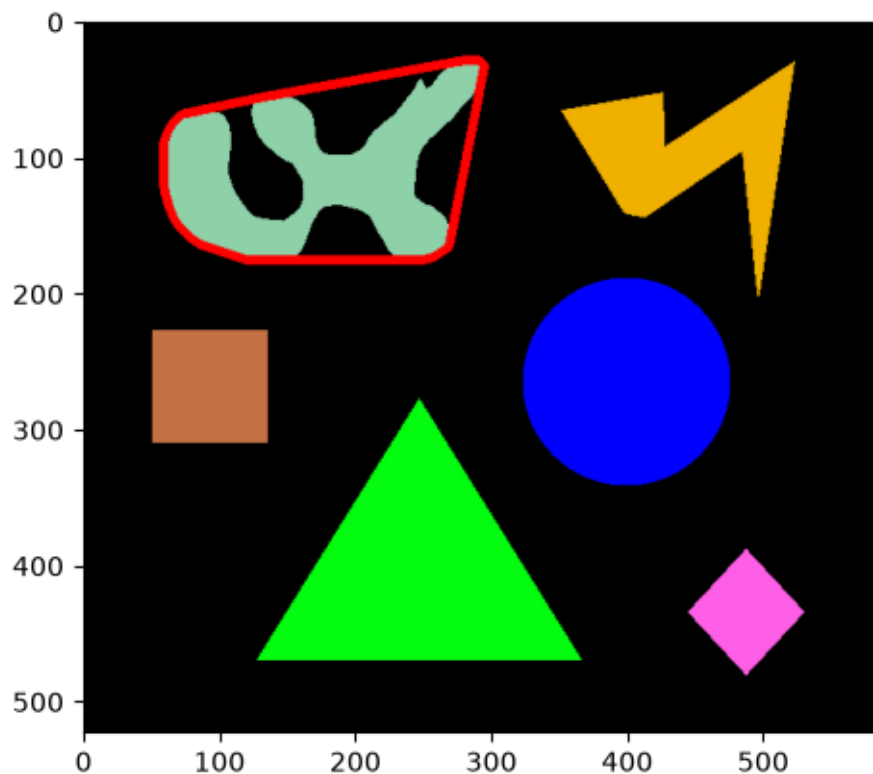
(28, 59, 176, 296)



```
[75]: # Y la convex hull image
from skimage.morphology import convex_hull_image

convex_image = convex_hull_image(objeto)
convex_image = convex_image.astype('uint8')
# plt.imshow(convex_image, cmap='gray')
# plt.show()

conts,_ = cv2.findContours(convex_image, cv2.RETR_TREE, cv2.
    ↳CHAIN_APPROX_SIMPLE) # Encontramos los contornos en una máscara
imagen = cv2.drawContours(img.copy(), conts, -1, (255,0,0), 5) # Dibujamos los
    ↳contornos
plt.imshow(imagen, cmap='gray')
plt.show()
```

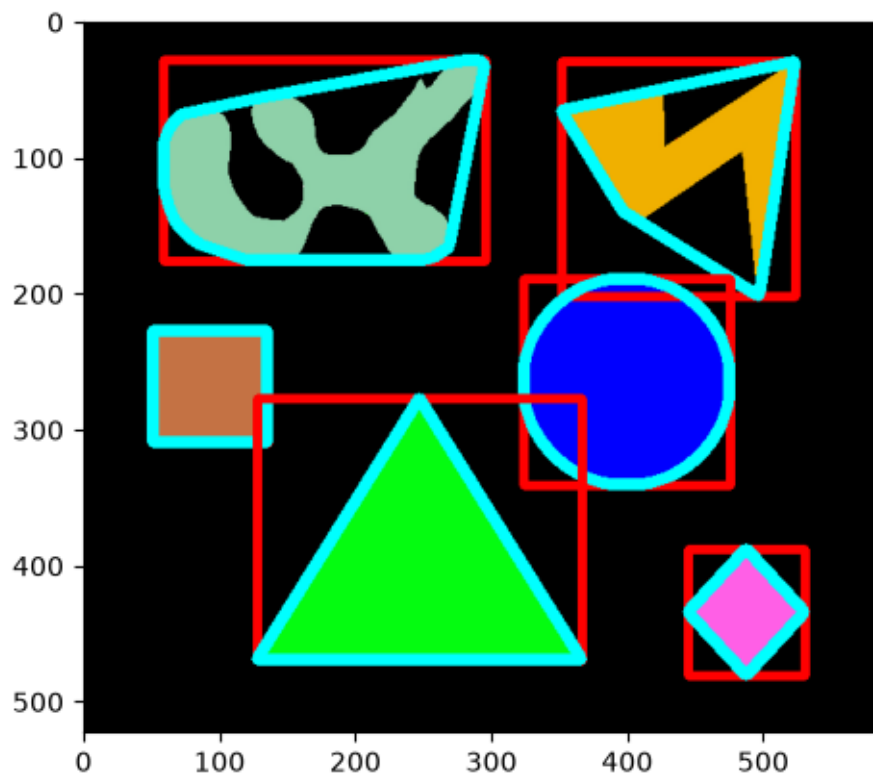


```
[76]: # Repetimos el proceso completo para todos los objetos de la imagen
imagen = img.copy()
for i in range(1, num+1):
    objeto = lab == i
    objeto = objeto.astype('uint8')

    # bbox
    prop = regionprops(objeto)
    bbox = prop[0].bbox
    imagen = cv2.rectangle(imagen, (bbox[1], bbox[0]), (bbox[3], bbox[2]), u
↪(255, 0, 0), 5)

    # convex hull
    convex_image = convex_hull_image(objeto)
    convex_image = convex_image.astype('uint8')
    conts, _ = cv2.findContours(convex_image, cv2.RETR_TREE, cv2.
↪CHAIN_APPROX_SIMPLE)
    imagen = cv2.drawContours(imagen, conts, -1, (0,255,255), 8)

plt.imshow(imagen, cmap='gray')
plt.show()
```



```
[79]: # Vamos a extraer diferentes descriptores geométricos que nos ayudarán a
      ↪ caracterizar los objetos.
      # pip install tabulate
      from tabulate import tabulate
      import math

      new_lab, new_num = label(mask, return_num=True)

      # Extraemos las características geométricas
      headers = ['', 'Raro', 'Rayo', 'Círculo', 'Cuadrado', 'Triángulo', 'Rombo']
      A,BB,CA,E,ED,EX,MA,MiA,OR,P,S,CO,R = ['area'], ['bbox_area'], ['convex_area'],
      ↪ ['eccentricity'], ['equiv_diameter'], \
      ['extent'], ['major_axis'], ['minor_axis'], ['orientation'], ['perimeter'],
      ↪ \
      ['solidity'], ['compactness'], ['rectangularity']

      for i in range(1,new_num+1):
          objeto = new_lab == i
          prop = regionprops(objeto.astype(np.uint8))
          A.append(np.round(prop[0].area, 4))
          BB.append(np.round(prop[0].bbox_area, 4))
```

```

CA.append(np.round(prop[0].convex_area, 4))
E.append(np.round(prop[0].eccentricity, 4))
ED.append(np.round(prop[0].equivalent_diameter, 4))
EX.append(np.round(prop[0].extent, 4))
MA.append(np.round(prop[0].major_axis_length, 4))
MiA.append(np.round(prop[0].minor_axis_length, 4))
OR.append(np.round(prop[0].orientation, 4))
P.append(np.round(prop[0].perimeter, 4))
S.append(np.round(prop[0].solidity, 4))
CO.append(np.round(4*math.pi*prop[0].area/prop[0].perimeter**2, 4))
R.append(np.round(prop[0].area/prop[0].bbox_area, 4))

my_data = [tuple(A), tuple(BB), tuple(CA), tuple(E), tuple(ED), tuple(EX), \
            tuple(MA), tuple(MiA), tuple(OR), tuple(P), tuple(S), tuple(CO), \
            tuple(R)]
print(tabulate(my_data, headers=headers))

```

	Raro	Rayo	Círculo	Cuadrado	Triángulo
Rombo					
-----	-----	-----	-----	-----	-----
area	16198	10282	18344	7055	23119
4018					
bbox_area	35076	29756	23104	7055	46127
7998					
convex_area	27405	16959	18488	7055	23307
4102					
eccentricity	0.8556	0.7941	0	0.2157	0.371
0.3866					
equiv_diameter	143.61	114.418	152.828	94.7771	171.569
71.5254					
extent	0.4618	0.3455	0.794	1	0.5012
0.5024					
major_axis	268.45	186.995	152.828	98.1428	195.774
76.1969					
minor_axis	138.975	113.644	152.828	95.8332	181.801
70.2735					
orientation	-1.4236	1.5298	-0.7854	1.5708	-1.5708
0					
perimeter	1029.77	736.02	503.245	332	717.962
255.588					
solidity	0.5911	0.6063	0.9922	1	0.9919
0.9795					
compactness	0.1919	0.2385	0.9102	0.8043	0.5636
0.7729					
rectangularity	0.4618	0.3455	0.794	1	0.5012
0.5024					

```

C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:18: FutureWarning:
`RegionProperties.bbox_area` is deprecated starting in version 0.26 and will be
removed in version 2.0. Use `RegionProperties.area_bbox` instead.
    BB.append(np.round(prop[0].bbox_area, 4))
C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:19: FutureWarning:
`RegionProperties.convex_area` is deprecated starting in version 0.26 and will
be removed in version 2.0. Use `RegionProperties.area_convex` instead.
    CA.append(np.round(prop[0].convex_area, 4))
C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:21: FutureWarning:
`RegionProperties.equivalent_diameter` is deprecated starting in version 0.26
and will be removed in version 2.0. Use
`RegionProperties.equivalent_diameter_area` instead.
    ED.append(np.round(prop[0].equivalent_diameter, 4))
C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:23: FutureWarning:
`RegionProperties.major_axis_length` is deprecated starting in version 0.26 and
will be removed in version 2.0. Use `RegionProperties.axis_major_length`
instead.
    MA.append(np.round(prop[0].major_axis_length, 4))
C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:24: FutureWarning:
`RegionProperties.minor_axis_length` is deprecated starting in version 0.26 and
will be removed in version 2.0. Use `RegionProperties.axis_minor_length`
instead.
    MiA.append(np.round(prop[0].minor_axis_length, 4))
C:\Users\USER\AppData\Local\Temp\ipykernel_1376\3802536251.py:29: FutureWarning:
`RegionProperties.bbox_area` is deprecated starting in version 0.26 and will be
removed in version 2.0. Use `RegionProperties.area_bbox` instead.
    R.append(np.round(prop[0].area/prop[0].bbox_area, 4))

```

[]:

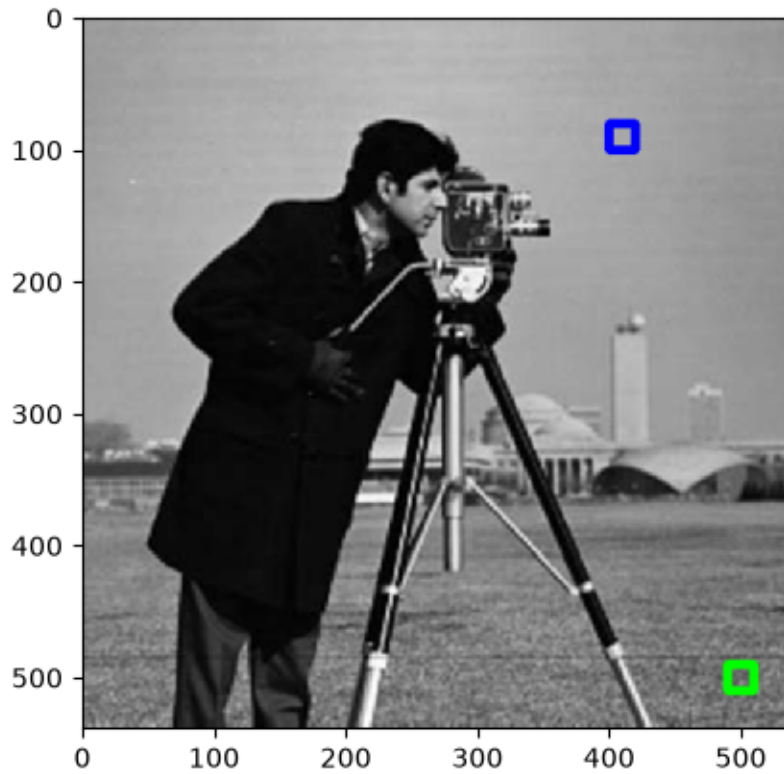
6 3_2_5_Descriptores_textura

```

[81]: # En primer lugar, obtenemos la máscara de la imagen original mediante la
      ↳ umbralización con Otsu
import matplotlib.pyplot as plt
import cv2
import numpy as np

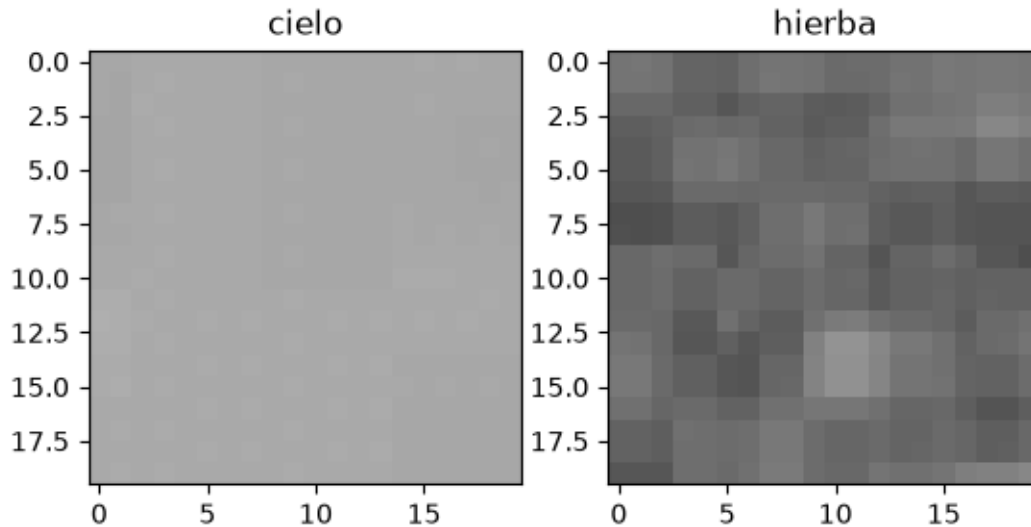
img = cv2.imread('images/photographer.png')
window = cv2.rectangle(img.copy(), (400,80), (420,100), (0, 0, 255), 5)
window = cv2.rectangle(window, (490,490), (510,510), (0, 255, 0), 5)
plt.imshow(window, cmap='gray')
plt.show()

```



```
[82]: # Seleccionamos una región del cielo y una de la hierba
cielo = img[80:100, 400:420]
hierba = img[490:510, 490:510]

fig, ax = plt.subplots(1,2)
ax[0].imshow(cielo, cmap='gray')
ax[0].set_title('cielo')
ax[1].imshow(hierba, cmap='gray')
ax[1].set_title('hierba')
plt.show()
```



6.1 MATRIZ DE COOCURRENCIA

```
[83]: # MATRIZ DE COOCURRENCIA (Gray-Level Cooccurrence Matrix - GLCM)
from tabulate import tabulate
from skimage.feature import graycomatrix, graycoprops # en versiones antiguas
↳ de skimage: greycomatrix, greycoprops

headers = ['', 'cielo', 'hierba']
CN, DIS, H, ASM, E, COR = ['contraste'], ['disimilitud'], ['homogeneidad'],
↳ ['ASM'], ['Energía'], ['Correlación']

patches = []
patches.append(cielo)
patches.append(hierba)

for i in range(len(patches)):
    image = cv2.cvtColor(patches[i], cv2.COLOR_BGR2GRAY)
    GLCM = graycomatrix(image, distances=[5], angles=[0], levels=256,
↳ symmetric=True, normed=True)
    CN.append(graycoprops(GLCM, 'contrast')[0,0])
    DIS.append(graycoprops(GLCM, 'dissimilarity')[0,0])
    H.append(graycoprops(GLCM, 'homogeneity')[0,0])
    ASM.append(graycoprops(GLCM, 'ASM')[0,0])
    E.append(graycoprops(GLCM, 'energy')[0,0])
    COR.append(graycoprops(GLCM, 'correlation')[0,0])

my_data = [tuple(CN), tuple(DIS), tuple(H), tuple(ASM), tuple(E), tuple(COR)]
print(tabulate(my_data, headers=headers))
```


	cielo	hierba
-----	-----	-----
contraste	3.68	333.393
disimilitud	1.4	13.8533
homogeneidad	0.512471	0.0921279
ASM	0.142717	0.00510556
Energía	0.377779	0.0714532
Correlación	0.190521	-0.0705127

6.2 Scale-Invariant Feature Transform

```
[84]: # SIFT algorithm
img = cv2.imread('images/photographer.png')

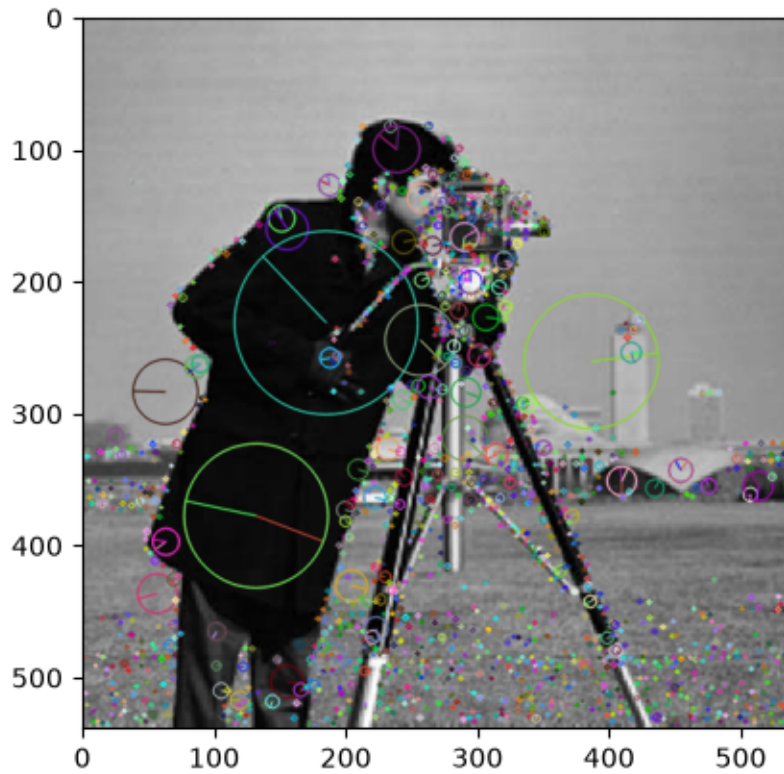
# Definición del algoritmo
sift = cv2.SIFT_create()

# Identificamos los puntos de interés
kp = sift.detect(img, None)

# Extraemos el descriptor para obtener el fingerprint
kp, des = sift.compute(img, kp)
print(np.shape(des))
# print(des) # Matriz donde las filas son los keypoints y las columnas las
↳ características de dichos puntos clave

# Visualizamos los puntos de interés
img = cv2.drawKeypoints(img, kp, img, flags=cv2.
↳ DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
plt.imshow(img)
plt.show()
```

(1672, 128)



6.3 Histogram of Oriented Gradients

```
[86]: # HISTOGRAM OF ORIENTED GRADIENTS
# En primer lugar, obtenemos la máscara de la imagen original mediante la
# umbralización con Otsu
from skimage import io

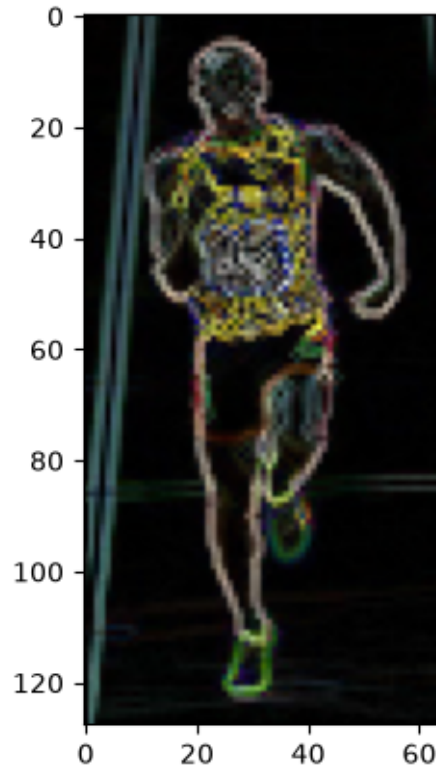
img = io.imread('images/pedestrian.jpg')
img = np.float32(img/255)
crop = img[75:550,475:700]
crop = cv2.resize(crop, (64, 128))
plt.imshow(crop, cmap='gray')
plt.show()
```



```
[87]: # Calculamos el gradiente con el kernel de Sobel
crop = np.float32(crop)
gx = cv2.Sobel(crop, cv2.CV_32F, 1, 0, ksize=1)
gy = cv2.Sobel(crop, cv2.CV_32F, 0, 1, ksize=1)

# Calculamos la magnitud y la dirección del gradiente
mag, angle = cv2.cartToPolar(gx, gy, angleInDegrees=True)

# Visualizamos la imagen obtenida en la magnitud del gradiente
mag = mag/np.max(mag)
plt.imshow(mag, cmap='gray')
plt.show()
```



```
[91]: # Cálculo del descriptor: Histogram of Oriented Gradients
from skimage.feature import hog

H = hog(crop, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2),
        transform_sqrt=True, block_norm="L1")
np.shape(H)
```

ValueError Traceback (most recent call last)

Cell In[91], line 4

```
1 # Cálculo del descriptor: Histogram of Oriented Gradients
2 from skimage.feature import hog
3
----> 4 H = hog(crop, orientations=9, pixels_per_cell=(8, 8),
      ↪ cells_per_block=(2, 2), transform_sqrt=True, block_norm="L1")
5 np.shape(H)
```

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\skimage_shared\utils.p :

```
↪587, in channel_as_last_axis.__call__.<locals>.fixed_func(*args, **kwargs)
584 channel_axis = kwargs.get('channel_axis', None)
586 if channel_axis is None:
--> 587     return func(*args, **kwargs)
```

```

589 # TODO: convert scalars to a tuple in anticipation of eventually
590 #       supporting a tuple of channel axes. Right now, only an
591 #       integer or a single-element tuple is supported, though.
592 if np.isscalar(channel_axis):

File ~\anaconda3\envs\MachineLearning\Lib\site-packages\skimage\feature\_hog.py
→171, in hog(image, orientations, pixels_per_cell, cells_per_block, block_norm
→visualize, transform_sqrt, feature_vector, channel_axis)
    169 ndim_spatial = image.ndim - 1 if multichannel else image.ndim
    170 if ndim_spatial != 2:
--> 171     raise ValueError(
    172         'Only images with two spatial dimensions are '
    173         'supported. If using with color/multichannel '
    174         'images, specify `channel_axis`.'
    175     )
    177 """
    178 The first stage applies an optional global image normalization
    179 equalisation that is designed to reduce the influence of illumination
(...) 184 shadowing and illumination variations.
    185 """
    187 if transform_sqrt:

ValueError: Only images with two spatial dimensions are supported. If using wit
→color/multichannel images, specify `channel_axis`.

```

6.4 Local Binary Patterns

```

[93]: # LOCAL BINARY PATTERNS (LBP)
from skimage.feature import local_binary_pattern

img = cv2.imread('images/photographer.png')

# Definimos los parámetros iniciales
R = 1          # Radio
P = 8*R        # Número de píxeles vecinos
epsilon = 1e-7 # Para normalizar el histograma

# Extraemos la imagen LBP
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
lbp_image = local_binary_pattern(img_gray, P, R, method='uniform')
a,b = np.unique(lbp_image, return_counts=True)

# Calculamos el histograma
lbp_image = np.uint8(lbp_image)
hist = cv2.calcHist([lbp_image.ravel()], [0], None, [P+2], [0, P+2])

# Normalizamos el histograma

```

```

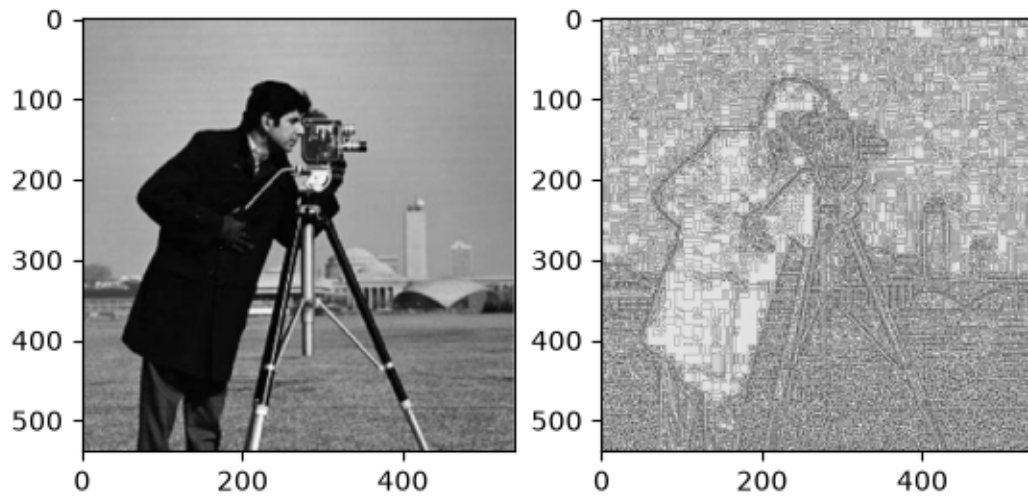
hist = hist.astype("float")
hist /= (hist.sum() + epsilon)

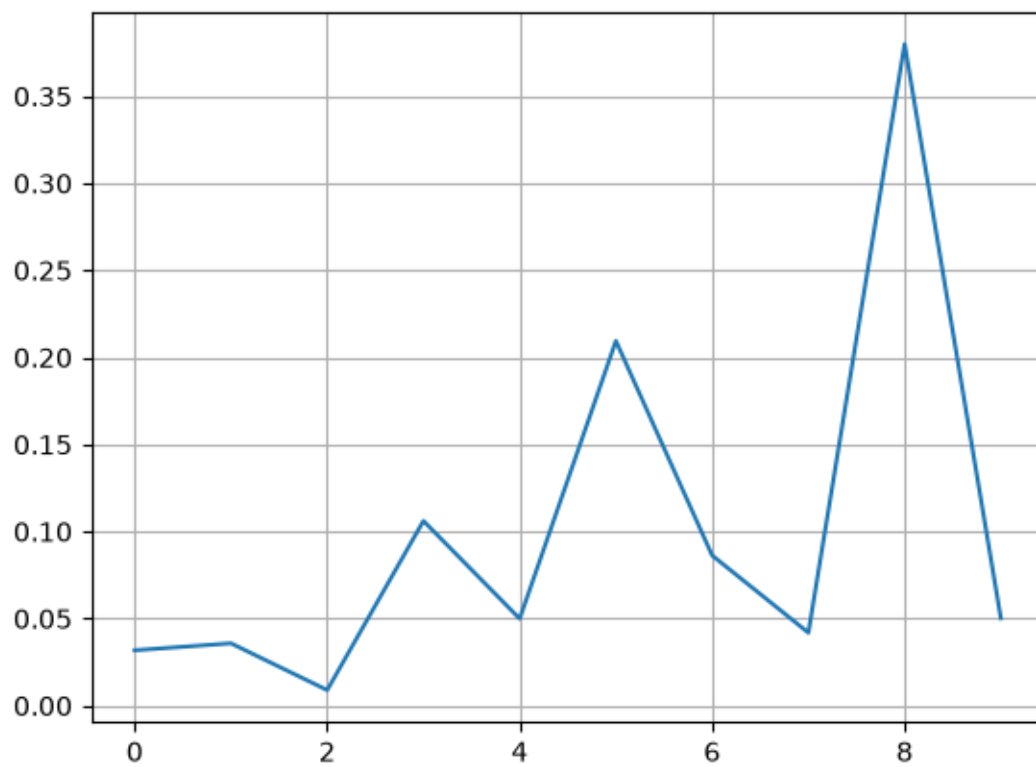
# Los P+2=10 bins del histograma serán los 10 atributos que nos permitirán
↪ caracterizar una imagen

# Visualizamos la imagen LBP
figs, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray')
ax[1].imshow(lbp_image, cmap='gray')
plt.show()

# Visualizamos el histograma
plt.plot(hist)
plt.grid(True)
plt.show()

```





[]: